ECHOCASE / ENGINEERING MASTERCLASS

从一条证据，到一个可靠的系统

Enterprise Full-Stack Engineering

with .NET & React

Clean Architecture · Domain-Driven Design
Data Consistency · React · Security · Reliability

DOMAIN → APPLICATION → EVIDENCE

基于真实应用与可复现实验的中文工程教材

2026.11 / REVIEW EDITION
miku-wwl/echocase

CONTENTS

目录

阅读路线与证据约定 9

怎样使用这本书 9

真实性与验收词汇 10

出版与工程的边界 10

环境和复现入口 10

第 1 章 需求、边界与可证伪的架构 11

从行为开始，而不是从目录开始 11

系统上下文与信任边界 12

为什么选择模块化单体 12

质量属性必须带场景 13

实验 E01：边界能否阻止退化 13

练习与参考答案 14

第 2 章 Clean Architecture 的机制与边界 15

依赖倒置究竟倒置什么 15

拆解上游模板而不复制答案 16

端口设计是一项业务选择 16

组装根与生命周期 17

不应把框架泄漏伪装成务实 17

实验 E02：证明结构与行为分别成立 17

练习与参考答案 17

第 3 章 DDD、聚合与调查状态机 18

领域模型的价值是缩小非法状态空间 18

聚合为什么包含这些子记录 19
解锁规则与正文保护 19
无向关联与规范化 19
结论、评估与不可逆动作 20
实验 E03：失败操作是否留下副作用 20
练习与参考答案 20

第 4 章 ASP.NET Core 请求管道与依赖注入 21

一次请求的成功由许多层共同决定 21
认证、授权和防伪并非同一检查 22
作用域与请求级工作单元 22
参数绑定与错误形状 22
案例：为什么数据库断开曾返回 500 22
实验 E04：观察中间件拒绝路径 23
练习与参考答案 23

第 5 章 CQRS、校验与应用编排 24

CQRS 可以从一个明确分工开始 24
校验应该放在哪里 24
显式编排的可追踪性 24
事务编排与领域行为的分界 25
取消、超时与错误传播 26
实验 E05：用失败证明用例是一个整体 26
练习与参考答案 26

第 6 章 REST 合同、条件写入与错误恢复 27

资源模型与动作语义 27
ETag 与 If-Match 是对用户意图的保护 27
状态码需要稳定的机器语义 28

生成类型解决什么，没解决什么 28

实验 E06：让合同漂移变成可见失败 29

练习与参考答案 29

第 7 章 EF Core、关系设计与迁移 30

关系模式表达第二道约束 30

集合映射与实体跟踪 31

查询快照与加载策略 31

迁移不是重新建库 31

实验 E07：数据库能否拒绝绕过领域的坏写入 31

练习与参考答案 32

第 8 章 乐观并发、事务隔离与冲突解释 33

丢失更新不是数据库写坏了，而是意图被覆盖 33

HTTP 预检查与数据库条件缺一不可 34

隔离级别保护不同范围 34

审计顺序也会影响冲突形状 34

实验 E08：先丢失，再保护 35

练习与参考答案 35

第 9 章 幂等、重试与提交结果不确定性 36

网络失败不等于业务失败 36

键、指纹和资源权限的顺序 37

原子回执比“先查再写”多了什么 38

原回执与最新状态有何不同 39

客户端必须保留同一次请求 39

实验 E09：重复、冲突与回滚 39

练习与参考答案 39

第 10 章 身份、资源授权与威胁建模 40

先明确攻击者可以控制什么 40
ASP.NET Identity 负责密码机制 40
资源授权必须靠可信身份 41
防伪令牌与同源策略 41
输出、日志与静态资源 41
实验 E10：负向安全路径 41
练习与参考答案 42
第 11 章 React 19 与可维护的前端边界 43
前端架构应沿着状态所有权划分 43
组件边界与实际规模 43
TypeScript 类型是沟通工具 44
表单与可访问性 45
登录成功但页面没变化的真实缺陷 45
实验 E11：刷新、导航和账户隔离 46
练习与参考答案 46
第 12 章 TanStack Query、乐观更新与冲突恢复 47
服务端状态的困难是时间，不只是位置 47
乐观更新不是提前宣布成功 47
412 的正确恢复方式 49
网络未知结果与明确失败 49
实验 E12：把乐观失败做成可复现操作 50
练习与参考答案 51
第 13 章 React Flow 与证据图谱的语义 52
图谱首先是业务模型的投影 52
本地节点位置与服务端关系分开 53
无向边不能被控件字段误导 53

交互限制与服务端规则的重复 53

性能从规模假设开始 53

实验 E13：同一关联能否被两种交互一致表达 54

练习与参考答案 54

第 14 章 单元、集成、架构与端到端测试 55

测试分层是为了解释失败 55

测试数据隔离与清理 55

并发测试需要独立上下文 56

浏览器测试验证用户可见结果 56

历史失败比“最后全绿”更有解释力 56

实验 E14：测试门禁是否真的会失败 56

练习与参考答案 57

第 15 章 SQL 执行计划、测量与性能回归 58

先问查询需要什么，再问索引建在哪里 58

如何读 EXPLAIN ANALYZE 58

测量设计与实际结果 58

API 测量是另一层证据 59

性能回归门禁如何避免噪声 59

实验 E15：计划变化能否解释时间变化 59

练习与参考答案 60

第 16 章 可观测性、依赖故障与恢复 61

可观测性从可回答的问题开始 61

存活与准备状态必须分离 61

真正停止依赖，与浏览器返回假错误不同 62

实测结果与时间口径 62

重试预算与故障放大 62

实验 E16：从症状到机制的排障 63

练习与参考答案 63

第 17 章 容器、CI 与 Azure 部署边界 64

一份可运行镜像包含哪些东西 64

重建与持久状态 64

CI 应执行什么 64

本地重建实验的限制 65

Azure 是部署设计，不是本机验证的延伸 65

实验 E17：删除可丢弃部分，验证持久部分 65

练习与参考答案 66

第 18 章 综合考核、证据审查与后续工程 67

综合任务：解释一个操作的一生 67

考核一：受控故障排查 67

考核二：维护性变更分析 68

考核三：审查一个“全绿”报告 68

发布验收如何分层 68

继续深化的优先级 68

练习与参考答案 69

附录 A 实验复现手册 70

E01 依赖边界退化 70

E02 领域脱离框架 71

E03 解锁与状态机 72

E04 请求管道保护 73

E05 失败原子性 74

E06 合同污染 75

E07 迁移与关系约束 76

E08 丢失更新与版本 77

E09 重复请求去重 78

E10 水平越权 79

E11 浏览器持久工作流 80

E12 乐观回滚 81

E13 图谱关联 82

E14 验收门禁自测 83

E15 查询计划优化 84

E16 真实依赖故障 85

E17 容器重建 86

E18 不可重复读 87

E19 写偏斜与可串行化 88

E20 幂等键误复用 89

E21 追踪上下文 90

E22 性能回归拒绝 91

E23 远程 CI 失败与恢复 92

E24 备份恢复机制 93

E25 移动端工作台 94

E26 出版校验器反例 95

附录 B 参考来源与工程词汇 97

一手技术来源 97

关键术语 98

复现结果不一致时 98

阅读路线与证据约定

这本书围绕一个可运行的应用展开：调查员登录 EchoCase，进入虚构案件“雾港档案”，查阅证据、建立关联、记录推理，最后提交由证据支持的结论。业务故事只是约束载体。真正要回答的是：当两个窗口同时修改调查时，谁的决定有效？网络中断后点击重试，会不会重复写入？隐藏按钮以后，别人的档案是否仍能被 API 读取？数据库已经提交而客户端没有收到响应时，什么才算完成？

读者应已熟悉一种后端语言、HTTP 和关系数据库。书中不展开语法入门，而是沿着同一调查工作流剖开框架机制，并把设计决定放回具体约束中评估。实现采用 .NET 10、ASP.NET Core、EF Core、PostgreSQL、React 19、TypeScript、TanStack Query 和 React Flow。这里的“企业工程”指可解释的约束、可审计的变更和可重复的验证，不表示系统已经承担过企业生产流量。

怎样使用这本书

推荐先运行应用，再从第 1 章逐步阅读。在领域和数据一致性章节，先预测实验结果，再执行命令，最后比较你的解释与参考答案。前端章节依赖已有的版本冲突与幂等语义；跳过后端协议直接写交互，通常会把正确性问题藏进缓存与按钮状态。

学习路径	重点章节	完成后应能解释
架构与领域	1–3、5	哪些规则属于聚合，什么条件值得增加抽象
API 与一致性	4、6–10	请求穿过哪些边界，如何处理并发、重复和越权
全栈交互	11–13	本地草稿、服务端快照与图谱布局为什么分开
质量与运行	14–18	什么证据足以支持一个结论，什么仍然没有验证

所有章节共享一份源代码，而不是为每个框架建立一个孤立演示。书中短代码片段在构建时直接摘取当前仓库文件；行号由发布源标记指定，超出文件边界会使构建失败。原始命令输出、测试结果和测量样本留在 `evidence/`，教材只展示有助于理解的部分。

真实性与验收词汇

真实执行表示真实进程、真实数据库和真实请求。**注入**表示人为引入的故障，例如拦截浏览器保存请求并返回 503，或短暂停止本项目的 PostgreSQL 容器。**合成数据**表示为了控制变量生成的数据，例如十万条所有者时间线记录。**设计分析**表示尚未执行的部署方案。四者不能互相替代。

验收使用 VERIFIED、PARTIAL、BLOCKED、NOT_RUN。VERIFIED 只覆盖列出的条件和证据；PARTIAL 表示某些条件已验证但整体范围未闭合；BLOCKED 表示当前存在明确外部障碍；NOT_RUN 表示没有运行。一个配置文件存在，不代表它已成功部署；本机数据库恢复，不代表具备跨区域灾难恢复能力。

当前后端结果为 38 / 38 项通过；浏览器结果为 4 项通过。这些数字来自最终结构化证据，而不是手动填写的成绩表。历史失败结果也保留在仓库中，用来说明缺陷怎样被测试发现及修正。阅读时应同时注意实验日期、运行环境和证据文件是否来自同一次验收。

出版与工程的边界

教材采用 Markdown → Pandoc HTML → Mermaid → Playwright PDF 的本地流水线，目录页码由 PDF 书签映射后再次打印。版面程序检查越界、空白与极小文字；人工视觉检查负责识别图意、断页和信息密度。自动检查不能证明每一句自然语言都正确，也不能替代独立读者的教学反馈。

本项目由同一执行者完成实现、实验和这份独立成册的验收材料。没有把自测描述成第三方审计。Azure 只给出迁移与部署设计，不创建付费资源。所有人物与事件均为虚构；教学评估分数仅对应固定案件规则，不评价现实中的司法判断。

环境和复现入口

从仓库根目录执行下列步骤。先确认 Podman 机器或 Docker 引擎已运行；数据库脚本只操作 `echocase-postgres`，不会清理其他项目。数据库卷保留调查档案，停止容器不会删除数据。

```
./scripts/dev-db.ps1
dotnet restore --locked-mode
./scripts/dev.ps1 migrate
./scripts/dev.ps1 api
# 在第二个终端
cd frontend
npm ci
npm run dev
```

第一次打开工作台需创建自己的本地账户。实验账户与合成案件用于学习，不包含真实案件资料。完整的测试、实验、容器运行、出版和清理命令见仓库 README 与 `docs/`。学习完成后保留你关心的证据，再按文档停用本项目进程。

第 1 章 需求、边界与可证伪的架构

从行为开始，而不是从目录开始

EchoCase 的需求不是“实现一个证据 CRUD”。调查员不能任意创建证据正文，不能在未调查两端时创建关联，不能用重复证据满足提交门槛，不能修改已封存结论。这些限制定义了一个有时间顺序、有所有权、有不可逆状态的业务流程。若先生成四层目录再补这些限制，架构很容易只剩分文件的形式。

把需求写成可观察行为，能立即改变验证方法。“系统支持并发”不可直接验收；“两个客户端基于版本 4 保存不同笔记，恰有一个成功，另一个收到 412，数据库只推进一次版本”可以验收。同样，“安全登录”过于宽泛；“没有防伪令牌的 Cookie 写请求被拒绝”是有明确威胁和观测的子目标。

业务模块	关键行为	不能被实现细节替代的约束
案件管理	浏览、开始、恢复个人调查	用户只能恢复自己的调查
证据管理	调查、解锁、持久保存发现	未调查的正文不出现在响应中
证据关联	建立、删除、图谱展示	两端已调查、无自环、无重复无向边
调查进度	笔记、复核、返回调查	状态和版本与子记录一致提交
结论提交	证据支持、固定评估、封存	先复核，至少三条不同的已调查证据

系统上下文与信任边界

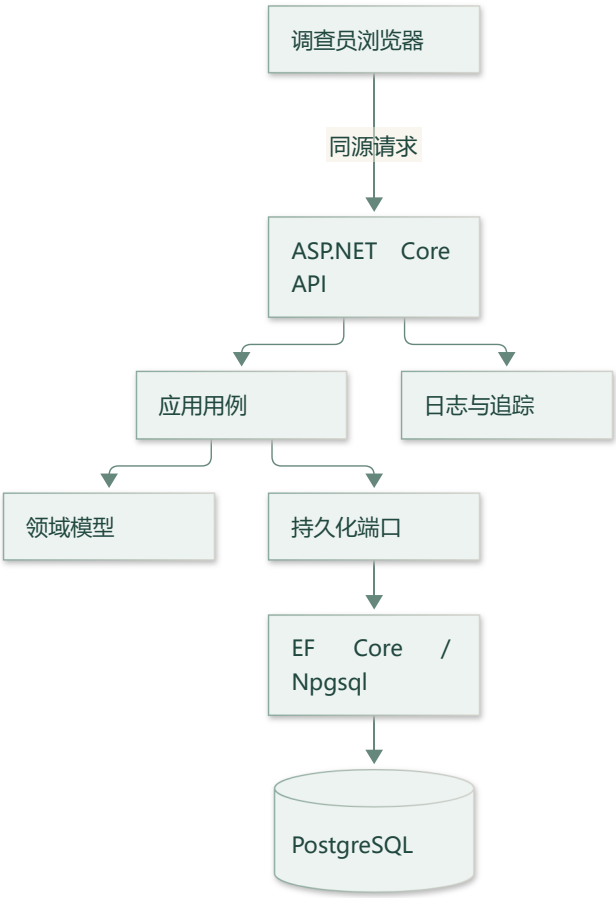


图 1-1　一次调查操作跨越的边界。图中的 PostgreSQL 是真实依赖；浏览器路由和 API 参数都属于不可信输入。

用户界面属于体验边界，API 属于安全边界，事务属于一致性边界。三者有重合，但职责不同。禁用“进入复核”可以减少误操作，却不能防止用户直接发送 HTTP 请求。数据库外键可以阻止关联指向不存在的发现记录，却不会理解“复核中不可编辑”这样的业务语义。一个正确设计让多层防线各自承担适合它的约束，并明确它们之间的空隙。

调查聚合拥有证据发现、关联、笔记和结论。身份信息由 ASP.NET Identity 管理，案件规则在代码目录中版本化。该划分避免把密码、案件定义和一次调查都装进巨大聚合。静态目录适合单案件教学；若案件以后由编辑人员动态发布，则需要独立案件版本和审阅过程，不能在运行中修改规则后仍声称旧调查可重现。

为什么选择模块化单体

本项目最困难的问题是同一调查内部的正确性，不是独立团队之间的部署耦合。一个进程、一个数据库和明确的内部边界，让“发现 + 版本 + 审计 + 幂等回执”可以使用同一个本地事务。把证据与调查拆成服务，会立刻增加跨服务授权、重复投递、消息顺序和补偿问题，却没有解决当前任何已经测量的瓶颈。

模块化单体并不意味着所有类型都能互相访问。Domain 不依赖数据库；Application 定义用例与端口；Infrastructure 实现持久化；API 是组装与协议边界。这里的模块不是未来微服务的承诺，而是当前变化原因的隔离。当需求变化时，应先观察哪一层必须知道它，再决定边界是否合适。

方案	当前收益	当前代价	本项目决定
单文件演示 API	很快运行	领域、协议和数据库难以独立测试	不采用
四层单体	可测试边界、单库事务、简单运行	需要维护端口和 DTO	采用
按功能垂直切片	用例定位清晰	公共规则可能分散	在应用层按用例使用其思想
多服务	独立部署和扩展	分布式一致性与运维成本	当前需求不支持引入

质量属性必须带场景

可维护性不是抽象地“易于扩展”，而是增加一种关联规则时，领域测试应先说明新约束，应用层只负责解析命令，持久化只在模式需要变化时修改。可靠性不是“服务永不出错”，而是数据库暂停时，存活检查仍可回答，准备检查和业务请求明确失败，恢复后已提交档案仍可读取。

性能也需要范围。十万条合成时间线的索引实验只能证明特定选择率与排序条件下计划发生了变化；不能据此宣称系统支持十万并发调查员。质量属性场景应写明刺激源、环境、被刺激部分、可观察响应和测量方式，这会自然排除许多装饰性的“企业级”承诺。

实验 E01：边界能否阻止退化

工程问题是：一个开发者为了省事，让 Domain 直接引用 Infrastructure，门禁是否会失败？假设是，源码边界检查与运行时程序集引用检查应分别发现不同形式的越界。

先运行现有架构测试，观察 Domain 只引用系统程序集。然后在临时副本中增加向外项目引用，运行质量门禁自测。故障只注入副本，不能把主工作区改坏再忘记恢复。通过条件不是“输出一段警告”，而是坏输入的进程退出码非零，正常输入为零。

这个实验能证明已声明的依赖规则正在执行，但不能证明所有业务职责都正确。例如一个不依赖 EF 的领域类型仍可能偷偷承担展示格式化。架构门禁保护可机器识别的结构，架构评审仍要检查变化原因与语义。

练习与参考答案

1-A　设计： 若新需求是“多人协作调查”，能否只给调查增加一个共享开关？

参考答案：不能。需要定义参与人角色、加入与移除、资源权限、操作归属、并发冲突语义和审计。单一 OwnerId 的查询过滤不再充分。应先明确协作模型，再判断是否需要独立授权策略；这仍不自动要求多租户或微服务。

1-B　评审： 一个方案通过增加消息队列解决所有幂等问题。你会要求什么证据？

参考答案：要求说明需要跨进程异步通信的具体用例、重复投递边界、消息与数据库提交的原子性、消费者去重策略及失败恢复。队列的存在不证明恰好一次业务效果；当前单库问题可以用更小的事务方案解决。

1-C　动手： 在临时副本中改变依赖方向，证明门禁失败，再运行正常门禁证明恢复。记录两次退出码。只有坏输入确实失败，这项实验才闭合。

第 2 章 Clean Architecture 的机制与边界

依赖倒置究竟倒置什么

依赖倒置没有消灭依赖。调查用例需要读取与保存状态，这是事实；倒置的是源代码层面对具体实现的知识。Application 声明 IInvestigationStore，Infrastructure 用 EF Core 实现它，API 在启动时把两者连接。运行时控制流仍然从应用进入数据库，编译时引用却指向应用定义的端口。

若接口原样暴露 DbSet、IQueryable 或 DbContext，那么目录上虽然有一层接口，应用仍然必须理解 EF 的跟踪、翻译和执行语义。本项目选择显式读写方法，端口返回用途明确的 SessionView；代价是需要维护一组映射，并接受这个小规模用例的专用接口。

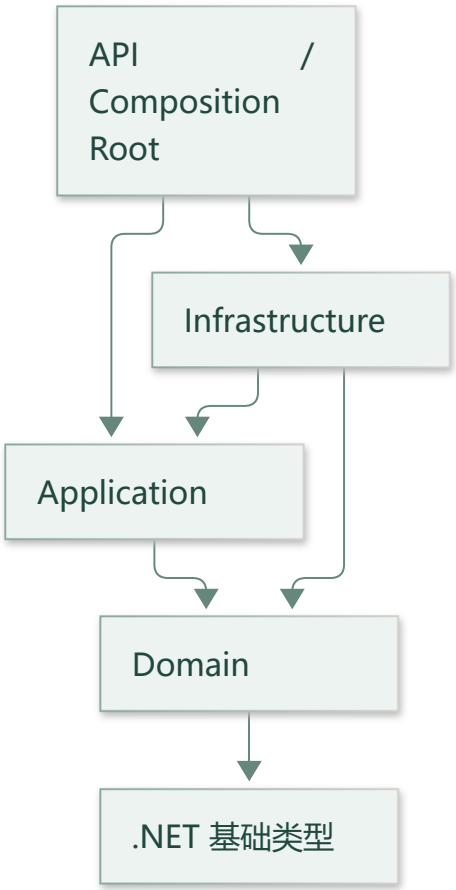


图 2-1　这是编译依赖图，不是请求流。Infrastructure 对 Domain 的引用用于映射，Domain 不知道这些映射存在。

拆解上游模板而不复制答案

Jason Taylor 模板将日志、授权、校验、性能等行为放进应用请求管道。它展示了横切关注点可以围绕处理器组合，而不在每个处理器中反复编写。这个机制依赖泛型请求、开放泛型行为注册与 DI 解析顺序。把接口名称搬到新项目，不会自动获得一致的执行顺序；绕过管道直接调用处理器也会绕过这些行为。

本项目研究了模板的授权行为、校验行为、依赖注册与 SavingChanges 域事件拦截器，保留向内依赖的思路，使用显式 InvestigationService 方法表达用例。当前规模下，一个服务与少量请求记录比引入通用总线更容易追踪。需要增加多个可组合行为时，可以再评估调度器；没有使用某个库，不代表没有 CQRS，也不代表该库没有价值。

模板的授权行为能够检查角色与策略，但它无法凭空知道“某个调查只属于当前用户”。资源权限仍必须在加载调查时使用可信用户 ID 过滤。另一个容易误读的机制是 SavingChanges：在保存前分发域事件，不意味着数据库已经提交。如果处理器在此时发送外部邮件，后续保存失败，外部副作用无法由数据库回滚。EchoCase 只把审计与业务数据写进同一事务，没有宣称实现可靠的外部事件总线。

端口设计是一项业务选择

```
{
    public int Status { get; } = status;
    public string Code { get; } = code;
}

public interface IInvestigationStore
{
    Task<IReadOnlyList<SessionView>> ListAsync(string ownerId, CancellationToken ct);
    Task<SessionView> GetAsync(Guid id, string ownerId, CancellationToken ct);
    Task<SessionView> WriteAsync(Change change, Func<Investigation?, Investigation> apply,
CancellationToken ct);
}
```

源码：src/EchoCase.Application/Investigations.cs，第 20-30 行。

WriteAsync 接收操作上下文和对聚合的变更函数。上下文包含可信所有者、幂等键、请求指纹、目标调查、预期版本和操作名称。这样，事务、回执和并发保护由同一个基础设施实现承载，领域变更仍由应用提供。它比“每张表一个泛型仓储”更接近当前真实的原子操作。

这个端口也有明确局限：回调是进程内抽象，不能直接用于远程调用；SessionView 的生成依赖当前案件目录；一个文件容纳全部用例只适合当前数量。用例增长后，应按功能拆文件、提取共享写策略，并维持同一门禁，而不是让服务演变成包含所有领域的控制中心。

组装根与生命周期

DbContext、持久化端口和用例服务都是请求级 Scoped 生命周期。同一请求共享跟踪与事务边界，不会把一个线程不安全的 DbContext 作为 Singleton 跨请求使用。TimeProvider 是 Singleton，因为系统时间源没有请求状态；测试可以替换它而不修改领域规则。

将服务注册集中到组装根还有一个审查收益：可以看清运行环境到底使用真实 PostgreSQL 还是替代实现。本项目集成测试使用 WebApplicationFactory 承载 ASP.NET Core，但持久化仍是 PostgreSQL。TestServer 替代的是 HTTP 宿主传输，不是数据库隔离语义。浏览器 E2E 再补上真实网络、Cookie 与 DOM 行为。

不应把框架泄漏伪装成务实

依赖 EF 的领域对象当然可以工作，但必须承认这种耦合。在 EchoCase 中，实体保持私有 setter 和只读集合视图，EF 通过基础设施映射访问字段。领域不增加数据库特性标注；关系约束留在映射中。这样做让纯领域测试不需要数据库，也让数据库映射的漏洞能由集成测试单独暴露。

另一方面，也不应为了纯净而重复实现密码哈希、HTTP Cookie 或 SQL 连接池。身份验证是成熟框架擅长的技术能力，放在边界上使用框架，比把这些细节包进几十个空壳接口更清楚。抽象的价值取决于它隔离了哪一种变化，而不是接口数量。

实验 E02：证明结构与行为分别成立

运行 CoreHasNoFrameworkOrOutwardDependencies，确认 Domain 没有外向程序集引用，Application 没有引用 Infrastructure、EF 或 ASP.NET。接着运行领域状态转换测试，确认在不启动 API 和数据库的条件下仍可验证业务规则。这两个结果分别回答“源代码知道什么”和“业务行为是什么”，不能互相替代。

再做一次维护性评审：假设允许第三种关联类型。列出需要修改的枚举、解析、数据库 CHECK、前端选项、颜色图例和测试。若只修改领域枚举后构建仍成功，不能认为变更已完成；生成契约与数据库约束是独立的兼容边界。这项分析应产出变更清单，不虚构尚未执行的扩展实验。

练习与参考答案

2-A： 为什么不直接把 IQueryable 返回给 API？参考答案：这会把查询形状、延迟执行和数据库翻译能力扩散到外层，资源过滤可能被遗漏，事务和查询成本也难以界定。需要更自由查询时，应定义明确的投影与筛选合同。

2-B： 如果在领域事件处理器里写审计，是否必然与业务一起提交？参考答案：取决于触发时机、DbContext 与事务是否相同，以及处理器是否引入外部副作用。接口名与“事件驱动”标签都不提供原子性保证。

2-C： 找出本项目一个值得保留的具体类和一个未来可能拆分的类。参考答案：Investigation 作为单一一致性边界值得保留；InvestigationService 在用例大量增长时可以按命令/查询拆分。拆分应保留共享安全与事务策略，而不是复制它们。

第 3 章 DDD、聚合与调查状态机

领域模型的价值是缩小非法状态空间

Investigation 不是装着公共属性的数据库记录。它通过 Start、Examine、Link、Review、Resume 和 Submit 表达业务行为，只在规则满足时改变状态。外部调用者无法直接设置 Score、Status 或 Version，也不能随意往只读集合中追加发现记录。这种封装不能阻止有数据库写权限的人直接修改表，但能让应用正常路径具有可检查的语义。

不变量分为结构不变量和历史不变量。关联两端不同、版本大于零是结构不变量；只能先调查门禁再解锁凭证册、先进入复核再提交结论，是依赖操作历史的规则。数据库约束擅长保护前者；领域状态机承担后者。设计时应逐条决定规则在哪一层被拒绝，以及失败后对象是否仍保持原状态。

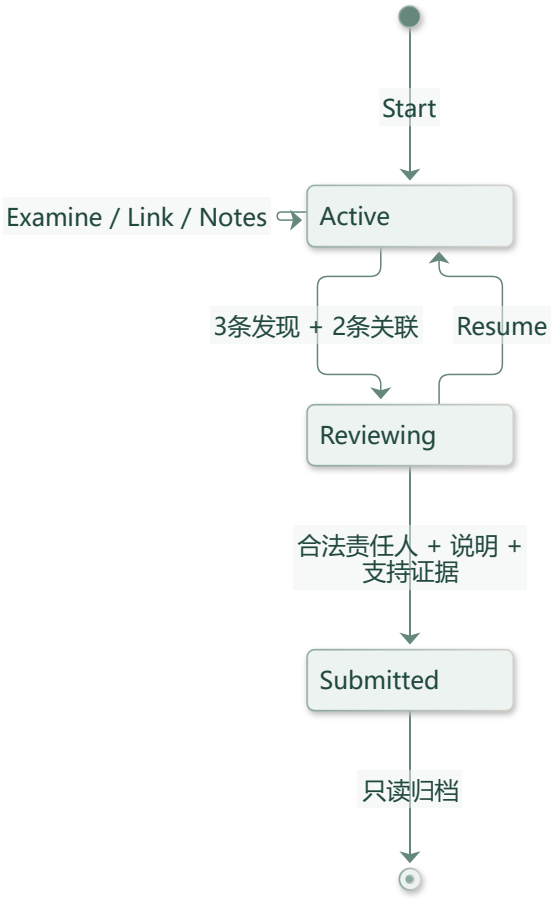


图 3-1　状态机没有从 Submitted 返回 Active 的边。若将来允许撤回，必须定义新的业务动作与审计含义。

聚合为什么包含这些子记录

结论是否可提交，依赖调查状态、发现集合和关联数量。它们必须在同一个一致性边界内判断，否则“读取三条证据→另一个操作删去关联→仍然提交”会破坏业务门槛。当前案件最多六条证据和十五对无向关联，聚合大小有明确上界，因此一次加载完整图并不构成无限增长的集合。

如果以后证据数增加到几万条，不能照搬当前全部加载的策略。可能需要独立关联聚合、预计算计数或明确的提交快照，但这会改变一致性证明。先测量聚合大小和操作竞争，再选择拆分方式；“聚合越小越好”与“所有相关数据都应在一个聚合”都过于绝对。

解锁规则与正文保护

案件目录定义前置证据关系。Examine 先检查状态可编辑，再检查证据存在、前置已发现和未重复调查，最后追加发现并增加版本。顺序很重要：先修改集合再检查规则，会让失败后的内存对象已经污染；即使数据库事务最终回滚，当前请求中的其他逻辑仍可能观察到坏对象。

```
    public void Examine(string evidenceId, DateTimeOffset now)
    {
        Editable();
        var e = CaseCatalog.Evidence(evidenceId);
        Require(IsVisible(e), "evidence_locked", "请先调查前置证据。");
        Require(!_discoveries.Any(x => x.EvidenceId == evidenceId), "already_examined", "该证据已调查。");
        _discoveries.Add(new(Id, evidenceId, now));
        Advance(now);
    }
```

源码：src/EchoCase.Domain/Investigation.cs，第 40-49 行。

响应投影把“可见摘要”和“已调查正文”分开。锁定证据返回通用摘要，未调查证据的 Body 为 null。仅在前端模糊正文、折叠卡片或用 CSS 隐藏内容，都不能提供访问控制，因为数据已经到达浏览器。这里仍然暴露了锁定证据的标题与类型，这是明确的产品选择；若要求连存在性也保密，就需要重新设计投影与解锁界面。

无向关联与规范化

“相互印证”与“存在矛盾”都是无向关系。领域用序数字符串比较固定两端顺序，再检查是否已有这一对证据。数据库唯一索引同样作用于调查 ID、规范化起点和终点。这样 A–B 与 B–A 不会变成两条重复边，替换关系类型也不会悄悄增加第二条互相冲突的边。

规范化只是存储策略，不是推理方向。图谱不应因为使用 React Flow 的 source/target 字段就画出因果箭头。前端已经移除箭头，保留线条和关系标签。这个例子说明：领域语言必须贯穿存储、API 和视觉表达，否则一个“只是画图”的细节也可能改变业务含义。

结论、评估与不可逆动作

Submit 要求复核状态、合法责任人、20–4000 字符说明和至少三条不同的已调查支持证据。说明长度只是防止空提交，不代表推理质量。确定性评分检查固定案件中的责任人与关键证据覆盖；它不是机器智能分析，也不是现实案件裁决。用相同输入得到相同结果，是当前教学模型可复现的前提。

提交后保留理由、支持证据与分数，并进入终态。若允许重复提交覆盖原结论，就会失去“这一次归档到底基于什么”的可审计性。当前选择是创建新的调查来重新尝试，而不是在同一封存档案中重写历史。

实验 E03：失败操作是否留下副作用

先对一个新调查尝试调查调度消息，预期得到 evidence_locked，Version 仍为 1。再调查门禁记录，预期凭证册可见，调度消息仍锁定。重复调查同一证据，预期被拒绝且版本不增加。最后完成三条发现和两条关联，进入复核，证明写笔记会失败，Resume 后可以继续。

这些测试不是对私有字段赋值后断言同一字段，而是从公开业务动作检验非法路径。更有价值的断言是“失败后没有变化”，它能发现检查与修改顺序的问题。数据库集成测试会进一步检查失败请求没有回执或审计残留，见第 9 章。

练习与参考答案

3-A： 为什么重复发现不直接忽略？参考答案：业务动作和传输重试是两件事。同一幂等键重试返回原回执；换一个新键再次调查同一证据，是新的业务请求，领域明确拒绝。也可以设计为无操作成功，但必须说明版本与审计的语义。

3-B： 可以把评分放在前端吗？参考答案：前端可显示预览，但归档结果必须由服务端根据受控规则计算，否则用户能伪造分数。评分规则仍不应出现在浏览器可用数据中作为隐藏答案。

3-C： 给“复核后还可以写笔记”提出两个设计方案。参考答案：允许非证据性备注作为独立、追加式审阅记录，或者要求 Resume 后再修改调查笔记。两者的审计含义不同；不能仅删除 Editable 检查而不讨论结论快照。

第 4 章 ASP.NET Core 请求管道与依赖注入

一次请求的成功由许多层共同决定

以“保存笔记”为例，客户端发出 PUT，请求先经过 Kestrel 的大小限制，再进入安全响应头和异常映射中间件。路由确定端点元数据，认证把 Cookie 票据转成 ClaimsPrincipal，限流按用户分区，授权确认已登录，防伪校验确认写请求携带匹配令牌，最后才执行参数绑定与应用用例。任何前置步骤失败，领域代码都不应被调用。

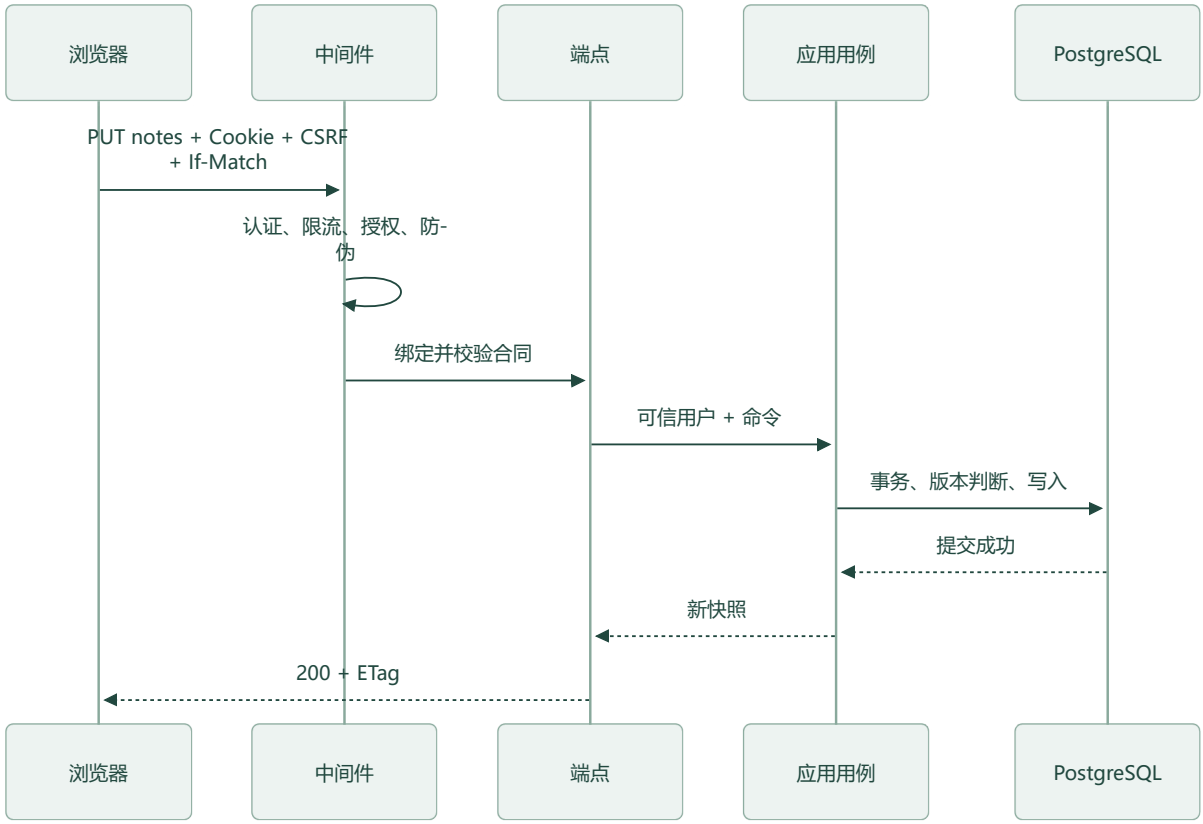


图 4-1　协议元数据在边界被解析；Cookie 和请求头不会进入领域模型。

中间件次序是一种安全决策。按用户限流之前必须完成认证，否则所有请求可能落入 anonymous 分区；需要读取端点策略的组件应在路由之后；异常映射需要包住可能抛错的后续逻辑。把所有安全组件注册进 DI 不等于它们已在正确位置执行。

认证、授权和防伪并非同一检查

认证回答“请求代表谁”；端点授权回答“这个端点是否要求已登录”；资源授权回答“这个用户能否操作这份调查”；防伪校验回答“携带浏览器自动 Cookie 的写请求，是否来自持有合法请求令牌的页面”。一个请求可能通过前三项中的部分检查，仍然因为另一项失败。

ASP.NET Core 10 对已识别 API 端点的 Cookie 认证失败采用 401/403 语义。本项目也显式设置跳转事件，使 API 合同容易审查：浏览器收到登录 HTML 不应被误认为有效的 SessionView。前端不能仅根据 fetch 没有抛网络异常就认为操作成功，必须检查 HTTP 状态和响应结构。

作用域与请求级工作单元

每个请求有一个 DI 作用域。EchoDbContext、PostgresInvestigationStore 和 InvestigationService 都是 Scoped；注入到同一请求中的实例共享 DbContext。事务由持久化端口开启，应用服务不手工创建第二个上下文。这种约束使审计和幂等回执能与聚合一起提交。

不要把 Scoped 服务捕获进 Singleton 后台对象。即使第一次调用看起来正常，作用域结束后对象可能已释放，或后续请求并发访问同一个上下文。真正的后台任务应显式创建作用域，并重新加载它需要的状态。EchoCase 当前没有后台任务，因此也没有为了演示而引入无用的后台队列。

参数绑定与错误形状

请求记录使用非空构造参数，HTTP JSON 配置拒绝缺失必需参数、显式 null 和未知成员。未知成员拒绝策略能提早暴露前后端合同漂移；代价是滚动部署时不能随意让新客户端向旧服务器发送新增字段。未来若需要宽容读取，应针对具体兼容策略调整，而不是一边声称严格合同，一边静默忽略拼错的属性。

业务错误由 DomainRule 表达，映射为 422。版本冲突为 412，缺少 If-Match 为 428，幂等键复用为 409。数据库依赖不可用为 503，未知错误为 500。响应使用 Problem Details，包含稳定的机器代码和追踪编号，不把堆栈、SQL 或连接信息送到浏览器。

案例：为什么数据库断开曾返回 500

最初错误处理只匹配顶层 NpgsqlException。实际故障实验中，EF 的执行策略把连接失败包装成 InvalidOperationException，内层才是 NpgsqlException，于是兜底分支把它映射成 internal_error。静态阅读容易忽略异常包装，真实断库则让问题立即出现。

修正只识别已知包装形态，不把所有 InvalidOperationException 都当成临时数据库故障。后者会掩盖程序错误并鼓励无意义重试。修正后重新执行相同停库实验，确认存活为 200，准备检查与业务读取为 503，恢复后原档案仍可读取。失败与修复证据分别保存在可靠性证据目录。

实验 E04：观察中间件拒绝路径

从已登录客户端发送没有 X-CSRF-TOKEN 的写请求，预期 400；携带正确防伪令牌但缺少 If-Match，预期 428；携带完整头部但版本过期，预期 412。这三个结果证明请求确实到达不同保护层，而不是“任何失败都返回 400”。结合追踪编号检查日志，确认没有正文或密码被写入请求摘要。

重现时使用项目集成测试，不要把真实密码粘到终端录屏里。API 日志记录方法、路径、状态、耗时和 trace ID；身份凭证不属于普通诊断日志。若以后增加请求体日志，必须先明确字段脱敏和数据保留策略。

练习与参考答案

4-A： 为什么健康检查分 live 和 ready？参考答案：进程仍可响应与能够完成依赖数据库的工作是不同状态。数据库短暂中断时反复重启进程，可能增加连接风暴；应先让流量感知不可用，再依据故障类型恢复。

4-B： 所有 500 都自动重试三次会发生什么？参考答案：可能重复写入、放大数据库压力并延长故障。需要判断操作是否可安全重试、是否具有幂等键、失败是否发生于提交之后，并设置超时与预算。

4-C： 在开发环境和容器 HTTP 实验中观察 Cookie 的 Secure 属性，再解释生产配置为什么不同。参考答案：本地回环 HTTP 需要 SameAsRequest；生产应通过 HTTPS 并使用 Always。不能把本地配置直接发布到公网。

第 5 章 CQRS、校验与应用编排

CQRS 可以从一个明确分工开始

命令表达“希望改变什么”，查询表达“需要看到什么”。EchoCase 的 StartRequest、LinkRequest、NotesRequest 和 VerdictRequest 是命令输入；CaseSummary 和 SessionView 是查询与命令完成后的投影。命令与查询仍然使用同一个数据库，没有读副本，也没有消息总线。这个规模下，读写职责分离已能提供大部分可读性收益。

应用服务负责把可信用户、协议预期和领域动作连接起来，不负责判定哪个人应该为虚构案件负责，也不负责生成 SQL。API 解析 If-Match，应用服务计算请求指纹，领域执行状态规则，基础设施保证原子提交。每层都知道足够完成职责的信息，避免任何一层成为“什么都知道”的对象。

校验应该放在哪里

校验问题	所需知识	主要位置
JSON 缺字段或显式 null	HTTP 合同	API 参数绑定
版本头格式是否合法	HTTP 条件请求	API
幂等键是否是 UUID	用例协议	Application
证据是否已经调查	聚合状态	Domain
调查是否属于当前用户	身份与资源	持久化查询边界
同一对关联是否重复	聚合与关系约束	Domain + PostgreSQL

重复校验不一定浪费。用户界面限制长度可及时反馈，领域重复检查长度可保护所有调用者，数据库长度限制可保护直接写入。这三层面对不同失败源。应避免的是无意识地把同一套复杂规则复制成三个互相漂移的实现。

显式编排的可追踪性

```
    public static CaseSummary[] Cases() => [new(CaseCatalog.Harbor.Id, CaseCatalog.Harbor.Title,
CaseCatalog.Harbor.Subtitle, CaseCatalog.Harbor.Synopsis, CaseCatalog.Harbor.Suspects,
```

```
CaseCatalog.Harbor.Evidence.Count)];
    public Task<IReadOnlyList<SessionView>> List(string owner, CancellationToken ct) =>
store.ListAsync(owner, ct);
    public Task<SessionView> Get(Guid id, string owner, CancellationToken ct) => store.GetAsync(id, owner,
ct);

    public Task<SessionView> Start(string owner, string key, StartRequest body, CancellationToken ct) =>
        Write(owner, key, null, null, "start", body, _ => Investigation.Start(owner, body.CaseId,
clock.GetUtcNow()), ct);
    public Task<SessionView> Examine(Guid id, string owner, string key, int version, string evidenceId,
CancellationToken ct) =>
        Write(owner, key, id, version, "examine", new { evidenceId }, i => { i!.Examine(evidenceId,
clock.GetUtcNow()); return i; }, ct);
    public Task<SessionView> Link(Guid id, string owner, string key, int version, LinkRequest body,
CancellationToken ct) =>
        Write(owner, key, id, version, "link", body, i =>
        {
            if (!Enum.TryParse<LinkKind>(body.Kind, false, out var kind) || !Enum.IsDefined(kind)) throw
new DomainRule("invalid_relation", "关联类型无效。");
            i!.Link(body.SourceId, body.TargetId, kind, clock.GetUtcNow()); return i;
        }, ct);
```

源码：src/EchoCase.Application/Investigations.cs，第 44-57 行。

处理 Link 时，应用首先把字符串解析为受控 LinkKind，拒绝未定义枚举值，然后调用领域 Link。这里没有把前端传入的 OwnerId 转交给领域；当前用户来自认证票据。启动调查时案件 ID 仍会被目录验证，不能因为前端只有一个选项就信任传入值。

用例服务通过 Write 统一计算指纹并进入事务端口。统一入口的收益是每个成功写入都遵循同一回执和审计策略。代价是开发者必须遵守入口约束；若以后新增端点直接注入 DbContext 写表，就会绕过保护。因此架构评审还应搜索新的数据库写入口，而不仅检查项目引用。

事务编排与领域行为的分界

领域回调只应修改当前聚合并返回它，不应执行外部网络调用、发送邮件或写另一套存储。事务端口可能因为并发冲突回滚，调用者也可能在提交结果不明时重试。回调含有外部副作用时，即使数据库事务完全正确，也无法证明整个用例原子。

这种限制让本项目可以用简单的本地事务完成当前需求。如果将来提交结论需要通知外部系统，应引入同事务的 outbox 记录，再由单独投递器至少一次发送并在接收端去重。那是新需求驱动的扩展设计，当前没有实现也没有假装验证。

取消、超时与错误传播

CancellationToken 从 HTTP 请求传到 EF 的异步调用，避免客户端断开以后继续执行不必要的长查询。取消并不意味着数据库一定没有提交，尤其是在 COMMIT 附近断开。若取消导致结果不确定，客户端仍应使用原幂等键确认，而不是生成新业务动作。

数据库命令超时用于限制一次数据库调用的等待，不等于整个用户操作的总预算。锁等待、连接建立和多个命令可能分别消耗时间。当前教学实现选择较短数据库命令超时，并保持失败明确；它没有实现自适应重试、熔断器或全局限流集群。描述范围时应把这些缺项说清楚。

实验 E05：用失败证明用例是一个整体

尝试在证据不足时进入复核。预期领域返回 insufficient_evidence，调查版本不变，数据库没有该幂等键的回执，审计行数仍然只有启动那一条。这个实验同时覆盖领域失败与事务回滚，回答“拒绝是否真的没有留下半个操作”。

再比较一次合法笔记保存：版本增加 1，聚合、审计和回执一起存在。不要仅断言 HTTP 为 200；用新 DbContext 查询数据库，并使用客户端随后的一次 GET 验证投影。新上下文排除了跟踪缓存导致的假成功。

练习与参考答案

5-A： 为什么不让命令只返回 204？参考答案：204 可以成立，但客户端随后需要再查询版本和投影。当前返回新 SessionView 与 ETag，方便明确本次提交后的状态；幂等重放返回的则是原始操作快照，可能已经落后于当前数据库。

5-B： 新增审计字段应放在领域事件里还是事务端口里？参考答案：取决于含义。业务事实可由领域表达；谁在什么请求中执行了什么操作属于应用审计。无论来源如何，都必须明确持久化原子性。当前审计不保存完整变更前后镜像。

5-C： 为“提交后给调查员发送邮件”画出可靠交付方案，并标注哪些部分当前没有实现。参考答案：业务事务写 outbox，独立投递器读取并重试，邮件适配器记录投递结果。不能把“事务内调用邮件 API”当成回滚保证。

第 6 章 REST 合同、条件写入与错误恢复

资源模型与动作语义

案件是可浏览定义，调查是某个用户拥有的持续资源。发现证据、进入复核和提交结论都是受业务约束的动作，不能简化成允许客户端任意 PATCH 状态字段。EchoCase 使用明确的动作端点，使服务端保留状态转换权；资源 ID 不携带所有权证明。

方法与路径	含义	写入条件
GET /api/cases	案件摘要	已登录
POST /api/investigations	开始调查	CSRF、幂等键
GET /api/investigations/{id}	获取个人调查	资源所有权
POST …/evidence/{id}/examine	调查证据	版本、幂等、领域规则
POST …/links	建立无向关联	两端发现、规范化去重
PUT …/notes	替换笔记内容	版本、防丢更新
POST …/state/review	进入复核	发现与关联门槛
POST …/verdict	提交封存	复核状态、支持证据

端点没有在路径中加入 v1，因为当前只有一个受控客户端，尚无并行支持多个公共 API 版本的需求。OpenAPI 文档名称 v1 是文档标识，不代表已经建立了完整的版本生命周期策略。若出现外部消费者，应定义兼容规则、弃用时间和契约测试，而不是只加一个 URL 前缀。

ETag 与 If-Match 是对用户意图的保护

GET 返回 ETag，例如 `"4"`。客户端保存时发送 If-Match: `"4"`，表明这次操作基于版本 4 的认识。服务器如果已经是版本 5，就不应悄悄覆盖。412 的意义不是“系统坏了”，而是“你依据的状态已过期，需要重新确认”。

当前实现只接受带双引号的正整数强版本标记，不接受星号、弱 ETag 或多个候选版本。这是有意收窄的应用合同，不能描述为完整实现 HTTP 条件请求规范。没有 If-Match 时返回 428，让遗漏保护的客户端尽早失败。

```
PUT /api/investigations/{id}/notes
Cookie: echocase.session=...
X-CSRF-TOKEN: ...
Idempotency-Key: 01937cd1-e567-4fbe-a964-a8b9b84e2a93
If-Match: "4"
Content-Type: application/json

{"notes":"门禁与凭证记录需要交叉核对。"}
```

示例中的凭证省略，UUID 只是格式示意，不是运行证据。实际请求头由客户端根据当前快照和一次用户意图生成。

状态码需要稳定的机器语义

状态	机器代码示例	客户端应做什么
400	csrf_invalid / invalid_request	刷新防伪上下文或修正合同
401	login_failed 或未登录	恢复身份，不重复业务动作
404	investigation_not_found	不继续尝试访问该资源
409	key_reused	这是不同请求，不能复用原键
412	version_conflict	刷新快照，保留草稿，让用户确认
422	evidence_locked / invalid_transition	解释业务规则，不自动重试
428	precondition_required	修复客户端缺少版本头的问题
503	database_unavailable	等待恢复，以原键确认不确定写入

问题详情供人理解，机器代码供程序分支。不要让前端用中文错误句子的子串决定恢复动作；文案改动不应改变正确性。追踪编号用于关联日志，也不应被当成用户身份或授权凭据。

生成类型解决什么，没解决什么

后端 OpenAPI 生成前端 schema.d.ts，客户端从 components 中引用 SessionView 等类型。它能在字段名称或数字类型变化时触发 TypeScript 编译问题。真实执行曾发现版本号被生成为 number | string；原因是 JSON 数字读取策略较宽松。后端改为严格数字后再生成，合同收敛到 number。

生成类型不能在运行时证明远端 JSON 正确，也不会自动添加 CSRF、If-Match 或幂等头。本项目的显式客户端承担这些协议行为。若引入不可信外部 API，可以再增加运行时 schema 校验；当前通过 API 合同负测试和 E2E 共同验证同仓库两端。

实验 E06：让合同漂移变成可见失败

发送 Notes 为 null 的请求，预期 400；发送额外 OwnerId，预期 400；用非法版本头请求，预期 428。然后从真实 API 再生成 schema.d.ts，与提交文件比较。若差异存在，CI 应失败，提醒同时提交客户端合同更新。

这个门禁需要格式稳定。生成文件不应再被自动格式化器改写，否则仅空格差异也会让合同检测失败。本项目把生成文件加入格式化忽略列表，并将契约再生成作为交付检查的一部分。

练习与参考答案

6-A： 412 与 409 是否可以互换？参考答案：当前合同把条件版本不满足表达为 412，把同幂等键指向不同请求表达为 409。两者恢复方式不同，混用会让客户端无法判断需要刷新状态还是修正键管理。

6-B： 返回原幂等回执后，是否可以直接把它当成最新调查？参考答案：不能。它证明原操作效果，不保证没有后续更新。客户端接收后还会使查询失效，重新获取当前快照；响应版本也可以防止盲目覆盖更晚缓存。

6-C： 给未来“下载调查报告”设计缓存规则。参考答案：先明确它是某个封存版本的不可变报告还是实时个人视图；个人实时 API 使用 no-store，静态且经过授权的版本化导出可以采用不同缓存策略。不能沿用公共静态资源的缓存配置。

第 7 章 EF Core、关系设计与迁移

关系模式表达第二道约束

领域规则不等于数据库约束。程序可能出现漏洞，迁移脚本也可能直接写表。因此 PostgreSQL 中仍保留关键 CHECK、唯一键和外键：Version 大于零；Status 位于合法集合；Submitted 与 Score 非空一致；关联不能自连；同一调查的一对证据只有一条关联；关联两端必须引用已经发现的证据。

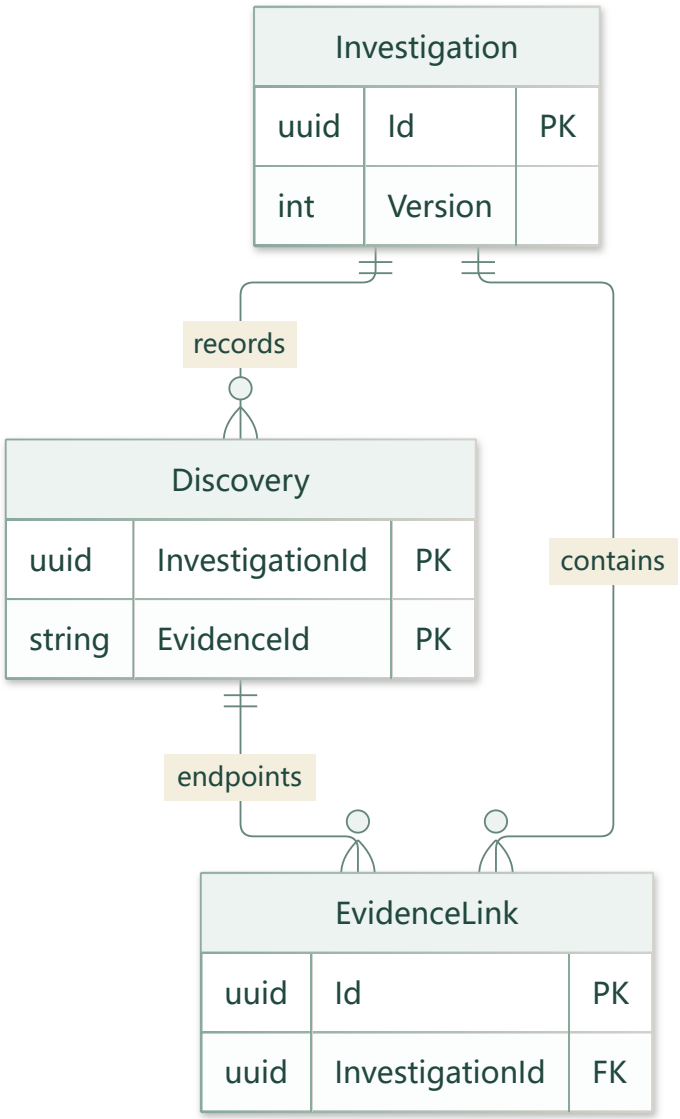


图 7-1　调查、发现和关联的核心关系。Identity、审计与回执省略，完整映射以 EchoDbContext 为准。

审计表当前没有对调查建立数据库外键，回执通过响应快照保留原操作结果；这两项不应被图中语义连线误认为完整参照完整性。核心调查、发现和关联的外键则由数据库执行。任何架构图都需要给出这种精度说明。

集合映射与实体跟踪

领域返回 IReadOnlyCollection，并在内部保留字段集合；EF 映射使用 Field 访问方式。这样查询后加载的发现和关联可以被跟踪，外部应用仍只能通过聚合动作改变它们。它隔离的是普通调用路径，不是阻止反射或拥有数据库权限的操作者。

EF 需要判断一个新关联到底应该 INSERT 还是 UPDATE。最初 EvidenceLink 在构造时生成 Guid，但主键映射沿用默认生成约定。在一个已跟踪集合中加入非默认主键对象时，EF 将它误判成已有实体，UPDATE 找不到行，抛出并发异常。修正将主键设为 ValueGeneratedNever，明确 ID 来自领域。

这个缺陷说明：单元测试通过不代表 ORM 映射正确。领域中的对象集合看起来完全正常，只有真实数据库工作流才暴露实体状态推断。遇到“没有并发用户却一直并发冲突”，应检查 ChangeTracker 状态与实际 SQL，而不是立即关闭并发检查。

查询快照与加载策略

同时 Include 两个集合可能形成笛卡尔扩张。AsSplitQuery 能减少重复行，但在 READ COMMITTED 下多个 SQL 语句可能看到不同提交时刻；若第一条读到旧版本，第二条读到新关联，返回值就不再是同一快照。当前聚合子项数量有固定上界，选择 AsSingleQuery 换取一次语句级快照。

这不是普遍建议。大集合可以使用只读投影、拆分查询加一致性事务、单独分页或明确弱一致性语义。应同时评估数据规模、隔离级别和用户能接受的状态差异。只用查询次数判断好坏，会忽略一致性与传输重复量。

迁移不是重新建库

初始迁移建立 Identity 与调查表；第二次迁移增加所有者与更新时间的组合索引。升级测试先应用初始迁移，插入身份和带笔记的调查，再应用最终迁移，用新上下文验证笔记保留，并确认至少两个迁移记录存在。它验证的是增量升级路径，而不是删除数据库后重新运行初始建表。

EF 工具生成新迁移后，需要重新编译再应用。执行中曾在生成新文件后直接使用 –no-build，导致运行中的迁移程序集仍旧，触发 PendingModelChangesWarning。正确修复是构建最新迁移，而不是照搬某些模板配置去忽略该警告。警告可能揭示真实未提交模式变化，应被认真处理。

实验 E07：数据库能否拒绝绕过领域的坏写入

在专用测试数据库里，直接向 Links 插入一条指向未发现证据的关联，预期 PostgreSQL 返回外键错误 23503。即使 Id、调查 ID 和关系类型都合法，两端发现不存在仍必须失败。随后用正常 API 完成发现并建立关联，证明合法路径没有被约束误伤。

实验还应检查事务回滚：先插入记录，再制造唯一键冲突，退出事务后第一条插入也不应残留。不要在日常开发库中运行破坏性 SQL；本项目脚本只创建随机命名的 echocase_lab_ 数据库，并在 finally 中清理这个精确目标。

练习与参考答案

7-A： 为什么证据目录没有全部存进数据库？参考答案：当前案件由代码版本固定，适合教学复现；动态编辑需求尚不存在。缺点是不能由管理员在线发布案件，也没有旧规则版本迁移机制。若案件演化，调查必须引用不可变版本。

7-B： 什么情况下 AsSplitQuery 更合适？参考答案：集合很大、单语句重复行成本高，并且可以通过事务快照或业务语义接受多语句读取时。必须证明返回快照的含义，而不只是观察查询变快。

7-C： 给“新增结论附注”设计兼容迁移。参考答案：先增加可空字段并部署能理解它的代码，写入验证后再决定是否收紧约束；不要在旧实例仍运行时立即加入它无法满足的 NOT NULL。Azure 真实滚动升级仍需额外验证。

第 8 章 乐观并发、事务隔离与冲突解释

丢失更新不是数据库写坏了，而是意图被覆盖

两个窗口读取值 0，各自计算加一，再依次写入 1，最终结果仍为 1。每次 SQL 都成功，数据库也忠实执行了两次赋值；被丢掉的是第二个操作之前没有看见第一个操作的事实。调查笔记的全量替换有相同风险，盲目“最后写入获胜”会覆盖用户尚未看到的内容。

版本方案让 UPDATE 带上原版本条件：只有 Version 仍为原值时才写入，并同时推进版本。第一个更新影响一行，第二个影响零行。EF Core 的并发标记把这个机制转换成 DbUpdateConcurrencyException；它不自动决定如何合并业务内容。

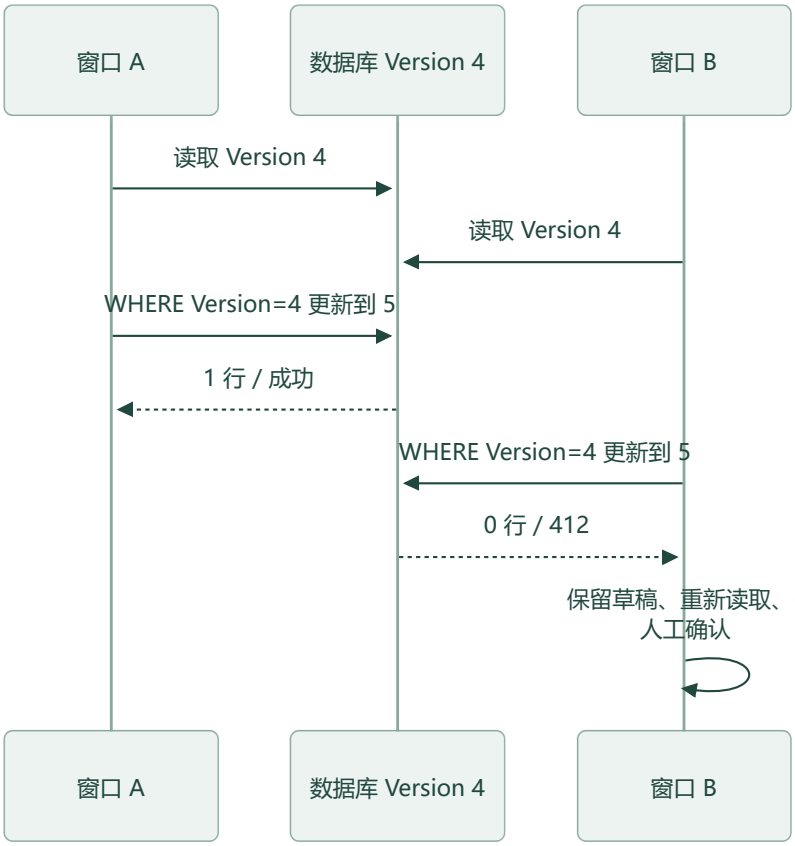


图 8-1　冲突是一条有价值的信息。自动换成版本 5 再提交，会重新引入覆盖用户意图的风险。

HTTP 预检查与数据库条件缺一不可

应用先比较 If-Match 与已加载 Version，可以及时拒绝明显过期请求；但比较之后、保存之前仍有竞争窗口。数据库 UPDATE 的原版本条件才是最终防线。只在 C# 中写一个 if，不能提供原子性，因为另一个事务可以在 if 与 SaveChanges 之间提交。

集成测试有两层证据。API 并发测试使用不同幂等键、相同预期版本，要求一方 200、一方 412；另一个测试用两个独立 DbContext 读取同一实体，先保存第一方，再确认第二方抛出 EF 并发异常。后者证明数据库并发令牌在工作，而不是仅靠入口串行化碰巧通过。

隔离级别保护不同范围

READ COMMITTED 的每条语句看到该语句开始时已提交的数据，因此同一事务两次 SELECT 可以看到不同值。REPEATABLE READ 在 PostgreSQL 中使用稳定事务快照，重复读同一行保持一致，但它不保证任意跨行不变量都成立。SERIALIZABLE 追踪可能无法串行化的依赖，使某些事务以 40001 失败，要求应用决定是否重试。

真实实验先把值设为 10。READ COMMITTED 事务先读 10，另一个连接写成 20，再读得到 20；REPEATABLE READ 得到 10、10。接着用“两名值班员至少留一人在岗”展示写偏斜：两个事务都看到两人，分别把自己设为离岗，在 REPEATABLE READ 下可以都提交，最终无人值班；SERIALIZABLE 下第二个写事务被 40001 拒绝，保留一人。

机制	能保护什么	不能自动解决什么
Version 条件更新	同一聚合的过期写	跨聚合不变量、业务合并
REPEATABLE READ	事务内稳定快照	一切跨行约束、写偏斜
SERIALIZABLE	可串行化依赖	重试策略、外部副作用
行锁或 advisory lock	被锁范围的竞争	未遵守锁协议的其他写路径

审计顺序也会影响冲突形状

EF 可能先插入审计记录，再执行聚合 UPDATE。两个事务争夺相同下一版本时，审计的唯一索引可能先报 23505，而不是聚合 UPDATE 报并发异常。最初 API 将这一特定冲突映射为 503，导致“一胜一冲突”的测试失败。

修正仅把已知审计版本唯一约束冲突映射为 412，其他唯一约束仍保留原错误含义。事务会回滚失败者的聚合、审计和回执。不能把所有 23505 都视为可重试版本冲突，否则会掩盖重复业务键或程序缺陷。

实验 E08：先丢失，再保护

执行 `experiments/database.py`，观察无保护计数器最终为 1，理论无丢失应为 2；版本条件写入的影响行数为 1、0。再执行真实 EF 双上下文测试与 API 竞争测试，观察相同机制在应用中的效果。三者分别回答 SQL 原理、ORM 映射和协议行为，形成闭合证据链。

该实验是受控交错，不是随机压力测试。优点是容易解释因果，缺点是不能发现所有时序。可以在理解机制后增加重复竞争轮次，但不应把某次没有冲突当作并发保护成功，也不应无理由反复运行已经通过的全套检查。

练习与参考答案

8-A： 收到 40001 能否直接重试数据库最后一条 SQL？参考答案：通常应重试整个事务，从新快照重新读取并判断业务规则；只重试最后一句会保留过期决策。若事务含外部副作用，还需要额外去重与交付设计。

8-B： 为什么同一调查的子记录修改也增加根版本？参考答案：提交结论依赖整个聚合。若只更新子表而不推进根版本，旧客户端无法发现它基于的证据图已变化。

8-C： 一个版本冲突是否应返回双方全部笔记？参考答案：可设计成三方合并界面，但必须保留基线、当前服务端和本地草稿。当前只保留草稿并刷新快照，不声称实现自动文本合并。

第 9 章 幂等、重试与提交结果不确定性

网络失败不等于业务失败

服务端可能已经提交事务，但响应在网络中丢失。客户端看到连接错误，无法区分“请求没到达”和“提交完成但响应没回来”。如果第二次点击生成新请求，可能创建两份调查或重复提交。禁用按钮只能减少同页双击，不能覆盖超时、代理重试、多进程实例与刷新页面。

EchoCase 为一次用户意图生成 UUID 幂等键，服务端以 OwnerId 与 Key 为作用域保存回执。请求指纹包含操作名称、调查 ID、预期版本和强类型请求内容。相同键相同指纹返回原响应；相同键不同指纹返回 409。失败请求不保存成功回执。

键、指纹和资源权限的顺序

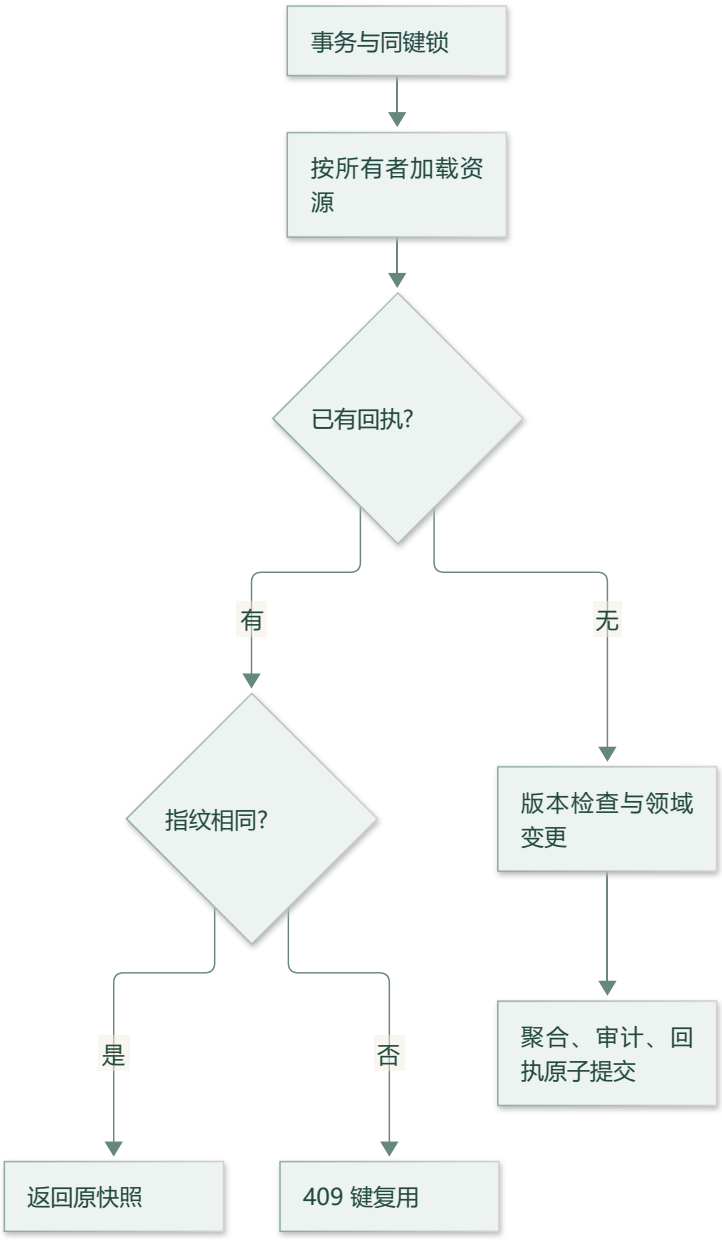


图 9-1　资源授权在回执重放之前执行，避免通过回执路径绕过所有权。相同键重放不再执行领域动作。

事务级 advisory lock 由 PostgreSQL 持有，随提交或回滚释放。它根据用户与键的哈希建立竞争范围，因此同一请求在多进程中仍可串行化。哈希碰撞只会让不相关请求暂时等待，不应导致回执混淆，因为实际查询仍使用完整 OwnerId 与 Key。

这个机制要求所有写路径都遵守协议。绕过端口直接写表，不能享有其幂等保证。数据库唯一索引作为另一道防线，但单靠唯一索引不能直接返回原业务响应，也不能说明相同键是否对应相同参数。

原子回执比“先查再写”多了什么

```
    public async Task<SessionView> WriteAsync(Change c, Func<Investigation?, Investigation> apply,
CancellationToken ct)
    {
        using var activity = Activities.StartActivity("investigation." + c.Operation);
        await using var tx = await db.Database.BeginTransactionAsync(ct);
        // A database-scoped lock, released on commit/rollback; unrelated keys proceed independently.
        var lockName = c.OwnerId + ":" + c.Key;
        await db.Database.ExecuteSqlInterpolatedAsync($"SELECT
pg_advisory_xact_lock(hashtextextended({lockName}, 0))", ct);
        Investigation? aggregate = null;
        if (c.SessionId is { } id)
            aggregate = await Graph.SingleOrDefaultAsync(x => x.Id == id && x.OwnerId == c.OwnerId, ct) ??
throw Missing();
        var previous = await db.Receipts.AsNoTracking().SingleOrDefaultAsync(x => x.OwnerId == c.OwnerId
&& x.Key == c.Key, ct);
        if (previous is not null)
        {
            if (previous.Fingerprint != c.Fingerprint) throw new ApplicationFault(409, "key_reused", "此幂
等键已用于其他请求。");
            Replays.Add(1);
            return JsonSerializer.Deserialize<SessionView>(previous.Response)!;
        }
        if (aggregate is not null && aggregate.Version != c.ExpectedVersion)
        {
            Conflicts.Add(1);
            throw new ApplicationFault(412, "version_conflict", "档案已被其他窗口修改。请刷新后重新确认操
作。");
        }
        var result = apply(aggregate);
        if (aggregate is null) db.Investigations.Add(result);
```

源码：src/EchoCase.Infrastructure/PostgresInvestigationStore.cs，第 25-48 行。

若先保存业务，再在另一个事务写回执，两个事务之间崩溃会留下“业务已完成、回执不存在”的窗口，重试仍会重复执行。若先写回执，再保存业务，则可能缓存一个从未完成的成功结果。当前方案把业务、审计与回执放在同一个 PostgreSQL 事务里，缩小到数据库本身的提交边界。

同一键的两个并发请求中，第二个等待第一个释放锁，随后在 READ COMMITTED 的新语句快照中看到已提交回执。测试要求两次响应版本相同，审计中该动作只有一条。不同键的请求不互相去重；它们通过聚合版本竞争，这是另一个正确性问题。

原回执与最新状态有何不同

回执存储操作完成时的 SessionView。若之后已有新操作，重放得到旧版本是正常现象：它回答“原操作是否完成”，不是“现在调查是什么”。前端完成写入后重新使查询失效，读取最新快照。若将来有更复杂的并发响应顺序，应进一步按版本防止旧响应覆盖更新缓存。

当前回执没有自动清理任务，因此保证只覆盖已保留的回执。文档建议在制定重试窗口后再添加保留策略；不能删除七天前回执后仍承诺永久幂等。存储完整响应也带来空间成本，规模扩大后可以评估操作结果引用或较小回执，但必须保留足够信息回答重试。

客户端必须保留同一次请求

自动重试关闭是刻意的。网络中断时，工作台提供“重试同一次操作”，复用原路径、正文、版本与键。用户修改草稿后再保存，属于新意图，应使用新键。把随机 UUID 生成放在每一次底层 fetch 重试里，会使服务端幂等机制失效。

CSRF 失效也不能一概当作业务重试。需要先恢复身份与防伪上下文，再判断原请求是否可能已经提交。如果服务器已经明确返回 400 且未进入业务处理，可以重新发起；如果连接在提交时断开，仍需以原键确认。

实验 E09：重复、冲突与回滚

运行三个负载不同的测试：重复启动调查得到同一资源；同键改变笔记内容得到 409；同键并发保存只增加一次版本和一条审计。再让领域规则失败，确认没有缓存该键；修正业务条件或版本后可以重用这次未成功的键。

实验不会证明恰好一次网络投递。它证明的是在同数据库、保留回执、遵守端口和身份作用域的条件下，重复请求产生一次已提交业务效果。清楚限定保证，比使用“Exactly Once”标签更有工程价值。

练习与参考答案

9-A： 为什么指纹包含预期版本？参考答案：版本是用户意图所依据的状态。相同正文基于不同版本可能是重新确认后的新操作；静默把它视为原请求，会混淆确认与重试。

9-B： 使用进程内字典保存键有什么缺陷？参考答案：重启丢失、多实例不共享、与数据库提交不原子，且没有明确保留与容量策略。它可以用于演示，却不能替代当前持久化保证。

9-C： 如果代理在服务端提交后丢弃响应，如何验证恢复？参考答案：在浏览器或代理层注入响应丢失，保留同一键重发，再从数据库确认版本和审计只推进一次。必须标注注入位置，并区分原始提交证据与客户端观测。

第 10 章 身份、资源授权与威胁建模

先明确攻击者可以控制什么

浏览器用户可以修改请求路径、JSON、HTTP 头和本地状态，也可以跳过页面直接调用 API。攻击者不需要看到“删除”按钮才能尝试删除别人的关联；随机 UUID 增加猜测成本，但不是授权机制。另一个站点可以诱导浏览器自动携带 Cookie 发起请求，因此仅验证 Cookie 也不足以保护写操作。

本项目威胁范围包含未登录读取、水平越权、正文提前泄漏、CSRF、过期状态覆盖、重复提交、登录爆破与参数合同污染。它没有实现对恶意数据库管理员的防护，没有外部身份提供者、MFA、邮箱确认、密码重置与组织级审计保留。把这些边界写出来，是为了让当前验证可被准确理解。

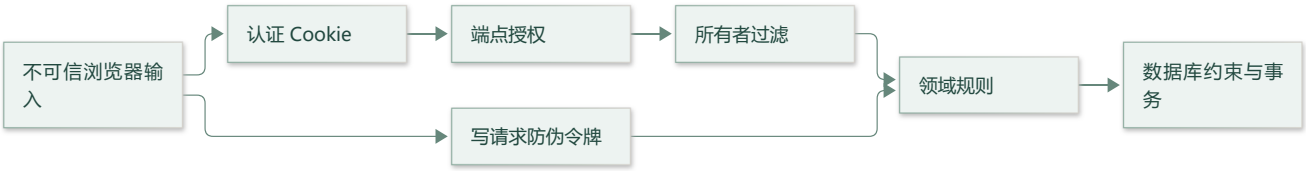


图 10-1　每道防线保护不同条件。任何一个框代表的机制都不能代替其他机制。

ASP.NET Identity 负责密码机制

EchoCase 使用 Identity 的用户管理与密码哈希，没有自创加盐算法，也不在配置中预置生产账户。注册要求唯一邮箱与至少 12 位、包含大小写和数字的密码。登录失败使用统一文案，按 IP 限流，并对已存在账户启用失败次数锁定。统一文案减少直接枚举，但当前没有证明响应时序无法区分存在与不存在的账户。

认证使用 HttpOnly Cookie，浏览器脚本无法直接读取票据；SameSite=Strict 限制跨站自动发送；生产使用 Secure Cookie。本地 HTTP 仅绑定回环地址，为教学运行采用 SameAsRequest。Cookie 过期为两小时，不滑动续期。服务端注销会清除浏览器 Cookie，但当前没有每次请求的安全戳重校验或集中会话撤销功能。

因此，在部署真实用户环境前，需要补齐账户生命周期、凭据轮换、会话吊销、身份提供者和恢复流程。这些是剩余工作，不因单机负测试通过就变成已实现能力。

资源授权必须靠可信身份

持久化查询同时使用 InvestigationId 和当前认证 OwnerId。找不到或不属于当前用户时统一返回 404，不向调用者确认目标是否存在。所有者从 ClaimsPrincipal 读取，JSON 不接受 OwnerId 字段。回执重放也先检查资源所有权，避免把幂等缓存变成越权捷径。

授权不只发生在 GET。建立关联、保存笔记、复核、删除关联和提交结论，都必须经过相同所有者加载路径。集成测试使用两个真实账户，验证第二个账户既不能读取，也不能修改第一个账户的调查。未来新增接口时，应把“读与写都受资源过滤”加入代码审查清单。

防伪令牌与同源策略

客户端先请求 `/api/auth/csrf`，服务器设置防伪 Cookie，并把请求令牌返回给同源脚本。写请求在 X-CSRF-TOKEN 头中携带它。登录后身份发生变化，客户端重新获取令牌；否则旧匿名令牌可能与当前身份不匹配。登录、注册和注销同样属于写操作，不应因为尚未进入业务模块就免除保护。

没有开放跨域 CORS。生产前端与 API 由同一容器提供，开发 Vite 使用代理保持浏览器同源。若未来拆开域名，需要重新评估凭证发送、SameSite、允许来源与防伪机制，不能用允许所有来源的配置解决开发错误。

输出、日志与静态资源

响应包含 nosniff、no-referrer、禁止嵌入的 CSP 等头部；脚本只允许同源，样式保留 unsafe-inline 是 React Flow 行内布局的实际需要，应在报告中说明。React 默认文本转义避免把调查笔记当 HTML；当前没有富文本编辑器，也没有使用 dangerouslySetInnerHTML 展示用户输入。

API 使用 no-store，避免个人档案被普通 HTTP 缓存保留。日志只记录请求摘要，不记录密码、Cookie、防伪令牌或笔记正文。追踪编号是诊断数据，不具有访问权限。容器重建时保留 Data Protection 密钥卷，使原认证票据可被同一应用继续解密；密钥卷的保护和备份仍属于运维责任。

实验 E10：负向安全路径

按顺序验证未登录 GET 返回 401、另一用户访问返回 404、带 Cookie 但缺防伪头的写请求返回 400、未知 JSON 成员被拒绝、未调查正文不出现在响应、连续无效登录最终被 429 阻断。每项检查都有独立断言，不能只用一次“登录成功”覆盖整个安全结论。

依赖扫描结果是另一个范围：当前 NuGet 和前端 npm 扫描没有报告已知漏洞，不等于没有零日漏洞、运行时漏洞或配置问题。首次 shadcn CLI 带来开发工具链漏洞；将不需要随应用保留的 CLI 移除，保留官方组件、所需 CSS 和 MIT 许可证后再扫描。没有使用强制降级去换取表面无告警。

练习与参考答案

10-A： 为什么防伪令牌不放 localStorage？参考答案：请求令牌可由内存持有并按身份刷新；localStorage 既不提供保密性，也增加跨会话陈旧状态。HttpOnly 保护的是认证 Cookie，不能以此推断所有脚本可读数据都安全。

10-B： 404 是否完全解决资源枚举？参考答案：它减少直接信息泄漏，但时序、列表接口、日志和其他侧信道仍需评估。当前测试只证明状态和资源过滤，不声称消除全部侧信道。

10-C： 将系统接入外部 OIDC 提供者前需要哪些决定？参考答案：可信发行者、回调与登出、主体映射、账户恢复、令牌存储和资源所有权迁移。继续保留调查资源授权；身份提供者不会自动知道 EchoCase 的所有权规则。

第 11 章 React 19 与可维护的前端边界

前端架构应沿着状态所有权划分

EchoCase 的页面包括登录、案件总览和调查工作台。工作台再分为证据、图谱、笔记、复核与结论。视觉上它们属于同一产品流程，代码上则需要区分服务端快照、用户草稿、视图选择和节点布局。把所有数据放进一个全局 store，会让保存成功、刷新、注销和切换调查的语义混在一起。

状态	所有者	保存位置	何时失效
当前用户	服务器会话	Query 缓存	注销、会话过期
调查快照	服务器	Query 缓存	成功写入、焦点刷新、定时刷新
尚未保存笔记	当前编辑者	组件本地状态	明确保存成功或主动放弃
当前调查 ID	导航	URL 查询参数	用户导航
图谱节点位置	当前视图	组件状态	离开视图
防伪请求令牌	会话上下文	内存	登录/注销后刷新

没有把认证票据放进 localStorage。URL 只携带调查 ID，不携带密码、令牌或证据正文。浏览器后退通过 popstate 恢复选中调查；刷新页面后重新加载服务器快照，而不是恢复一个过期的本地 JSON 副本。

组件边界与实际规模

当前 App 负责身份切换和页面导航，Dashboard 负责列表与启动，Workspace 负责一份调查的查询和写入生命周期，EvidenceGraph 负责图谱交互，Verdict 负责复核与提交表单。客户端协议集中在 api/client.ts，生成类型在 api/schema.d.ts，官方 shadcn 组件位于 components/ui。

Workspace 仍是较大的组件，这是当前维护性限制。继续增长时，应把证据面板、笔记编辑器和写入 hook 拆为独立模块，并用已有 E2E 保持行为。不能为了立刻达到“组件数量看起来合理”而拆出只有转发属性的空壳；应围绕状态生命周期与交互变化拆分。



图 11-1　真实浏览器中的案件总览。统计来自当前账户，案件插图为代码绘制的示意图。

TypeScript 类型是沟通工具

客户端直接引用 OpenAPI 生成的 DTO 类型，减少两端各自维护同名接口的漂移。严格 TypeScript 检查阻止把可能为空的正文直接当成字符串，也曾发现版本号数字合同过宽。类型系统不能验证网络实际返回的数据，也不能证明用户有权限；它帮助开发者在编译期发现一类错误。

请求函数把协议头、Cookie 发送、错误解析和网络失败集中起来。业务组件不拼接密码票据，不直接写 fetch 后忽略 response.ok。把网络错误包装为带状态、机器代码与 trace ID 的 ApiError，使工作台能为不同失败提供不同恢复路径。

```
export async function request<T>(
  url: string,
  options: {
    method?: string;
    body?: unknown;
    key?: string;
    version?: number;
  } = {},
): Promise<T> {
  const method = options.method ?? "GET";
```

```
  if (method !== "GET" && !csrf) await refreshCsrf();
  const headers: Record<string, string> = { Accept: "application/json" };
  if (options.body !== undefined) headers["Content-Type"] = "application/json";
  if (method !== "GET") headers["X-CSRF-TOKEN"] = csrf!;
  if (options.key) headers["Idempotency-Key"] = options.key;
  if (options.version !== undefined)
    headers["If-Match"] = `"${options.version}"`;
  let response: Response;
  try {
    response = await fetch(url, {
      method,
      headers,
      credentials: "same-origin",
      body:
        options.body === undefined ? undefined : JSON.stringify(options.body),
    });
  } catch {
```

源码：frontend/src/api/client.ts，第 23-49 行。

表单与可访问性

邮箱、密码、笔记、证据选择和结论说明都有可关联的标签；错误使用 role=alert，保存完成使用 role=status；选项卡具有 tablist、tab 与 aria-selected；图谱提供选择框建立关联的替代路径，不要求用户只能拖拽。这里验证了可通过标签定位的自动化流程，但没有完成全套屏幕阅读器和 WCAG 审计。

shadcn 组件生成到仓库，应用拥有修改责任。生成后的 Button 基于 Base UI，不能假设所有按钮默认都是表单提交按钮；登录和结论提交明确设置 type=submit。依赖组件库不免除理解原生表单行为的责任。

登录成功但页面没变化的真实缺陷

首轮浏览器测试中，注册和登录 HTTP 都成功，页面仍停在登录界面。原因是登录回调先清空整个 QueryCache，再用 setQueryData 创建新的 me 查询；现有观察器仍关联被移除的旧查询，未收到新数据。后端测试无法发现这个浏览器缓存生命周期问题。

修正保留 me 查询，只移除其他账户相关查询，再更新当前用户数据。注销采用同样原则，并刷新防伪令牌。这个案例比“调用 invalidateQueries 就能解决缓存”更具体：要理解观察器、查询实例和缓存键的关系，而不只是记住某个方法名称。

实验 E11：刷新、导航和账户隔离

创建账户后开始调查，保存笔记，记录 URL，刷新页面，再进入笔记页验证内容。注销后用另一个账户登录，确认前一个账户的调查不在列表中。当前后端越权测试和缓存清理共同保护该边界；不能依赖其中一方掩盖另一方遗漏。

移动端实验使用 390×844 视口，执行登录、启动、调查与图谱展示，并断言文档没有横向溢出。它证明这个具体视口与浏览器下的行为，不等于所有移动设备、缩放比例与辅助技术都已验证。

练习与参考答案

11-A： 为什么当前调查 ID 适合放 URL，草稿不适合？参考答案：ID 便于刷新和导航恢复，草稿可能敏感、很长且未提交；放 URL 会进入历史与日志，也没有清晰并发语义。

11-B： 本地布局是否必须保存到服务器？参考答案：取决于产品需求。当前明确提示布局只在视图保留，关联才是持久业务数据。若增加布局保存，应把个人呈现偏好与调查业务版本分开，避免移动节点造成无意义冲突。

11-C： 为工作台提出一项有依据的重构。参考答案：把笔记的 dirty、服务端基线和保存 mutation 提取为专用 hook，同时保留冲突与回滚 E2E。只拆 JSX 而不明确状态所有权，收益有限。

第 12 章 TanStack Query、乐观更新与冲突恢复

服务端状态的困难是时间，不只是位置

缓存中的 SessionView 是某次读取的快照。服务器可能已经变化，另一个窗口可能正在提交，当前窗口也可能有未保存草稿。把服务端快照复制进本地 useState，再让两者互相同步，常会产生看似随机的覆盖。设计前先明确每次同步发生的条件。

Query Key 包含资源种类、用户 ID 和调查 ID。用户 ID 不是后端授权凭据，但能防止不同账户的本地查询碰撞。工作台配置焦点刷新和定时读取，使其他窗口的已提交变化可以被看见；这些刷新不能覆盖 dirty 草稿。

乐观更新不是提前宣布成功

笔记保存时，onMutate 先取消当前查询，再保存旧快照，并在缓存中放入预期笔记。onError 恢复旧缓存；本地编辑器仍保留用户输入。onSuccess 采用服务器返回的版本和快照，清除 dirty；onSettled 使调查与列表失效，重新读取当前状态。

```
const mutation = useMutation({
  mutationFn: (op: Operation) =>
    request<Session>(`/api/investigations/${id}/${op.route}`, {
      ...op,
      method: op.method ?? "POST",
    }),
  onMutate: async (op) => {
    await cache.cancelQueries({ queryKey: qk });
    const previous = cache.getQueryData<Session>(qk);
    if (op.route === "notes" && previous)
      cache.setQueryData(qk, {
        ...previous,
        notes: (op.body as { notes: string }).notes,
      });
    return { previous };
  },
  onError: (_error, _op, context) => {
    if (context?.previous) cache.setQueryData(qk, context.previous);
  },
```

```
    onSuccess: (s, op) => {
      cache.setQueryData(qk, s);
      if (op.route === "notes") {
        setDirty(false);
        setSaved(true);
      }
    },
    onSettled: () => {
      void cache.invalidateQueries({ queryKey: qk });
      void cache.invalidateQueries({ queryKey: ["sessions"] });
    },
  });
  const s = query.data;
  const act = (route: string, body?: unknown, method?: string) => {
    if (s)
```

源码：frontend/src/App.tsx，第 527-560 行。

取消查询是为了避免较早发出的 GET 在乐观更新之后返回并覆盖缓存。它不取消其他客户端，也不让写请求获得独占权。版本冲突仍由服务器判断。由于当前只对笔记采用乐观更新，解锁证据和封存结论必须等待服务端确认；不同业务动作可以选择不同反馈策略。

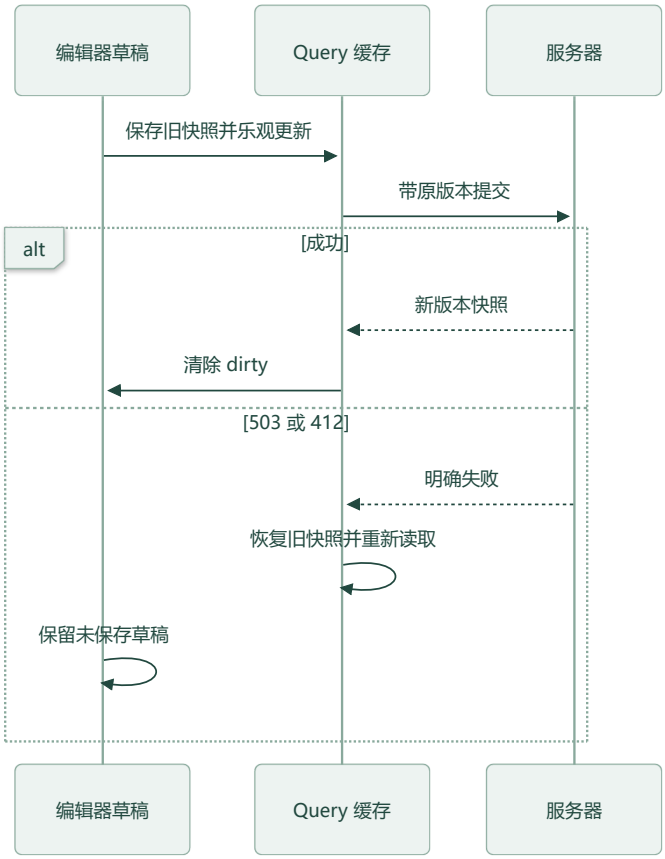


图 12-1　缓存回滚与草稿保留同时成立。回滚不是把用户刚输入的文字删掉。

412 的正确恢复方式

两个窗口都基于版本 1 编辑。A 先保存，B 收到 412。B 应看到服务端已变化的提示，草稿保留，重新读取后由用户确认是否提交它。自动把 B 的请求版本改为 2 再发送，会把“检测到冲突”变成“自动覆盖 A”，失去条件写入的意义。

当前工作台采用保留草稿与刷新快照的简单方案，未提供三方差异合并界面。重新点击保存代表用户基于新的服务端状态再次确认，但 UI 尚未展示完整差异，因此这是已披露的产品改进点。对长文档协作，单靠一个告警通常不够。

网络未知结果与明确失败

明确的 422 或 412 表示该请求没有成功提交，客户端应修正业务条件或重新确认。网络异常可能没有收到任何响应，结果不确定；界面允许复用 mutation.variables 中的原请求重试。这保留同一幂等键与版本，不会生成另一个启动或保存动作。

查询默认允许一次自动重试，写 mutation 默认不自动重试。读写风险不同，不应把同一个 retry=3 应用到所有请求。用户看到的“加载失败”和“保存结果未知”也应使用不同文案，因为后者需要保护原意图。

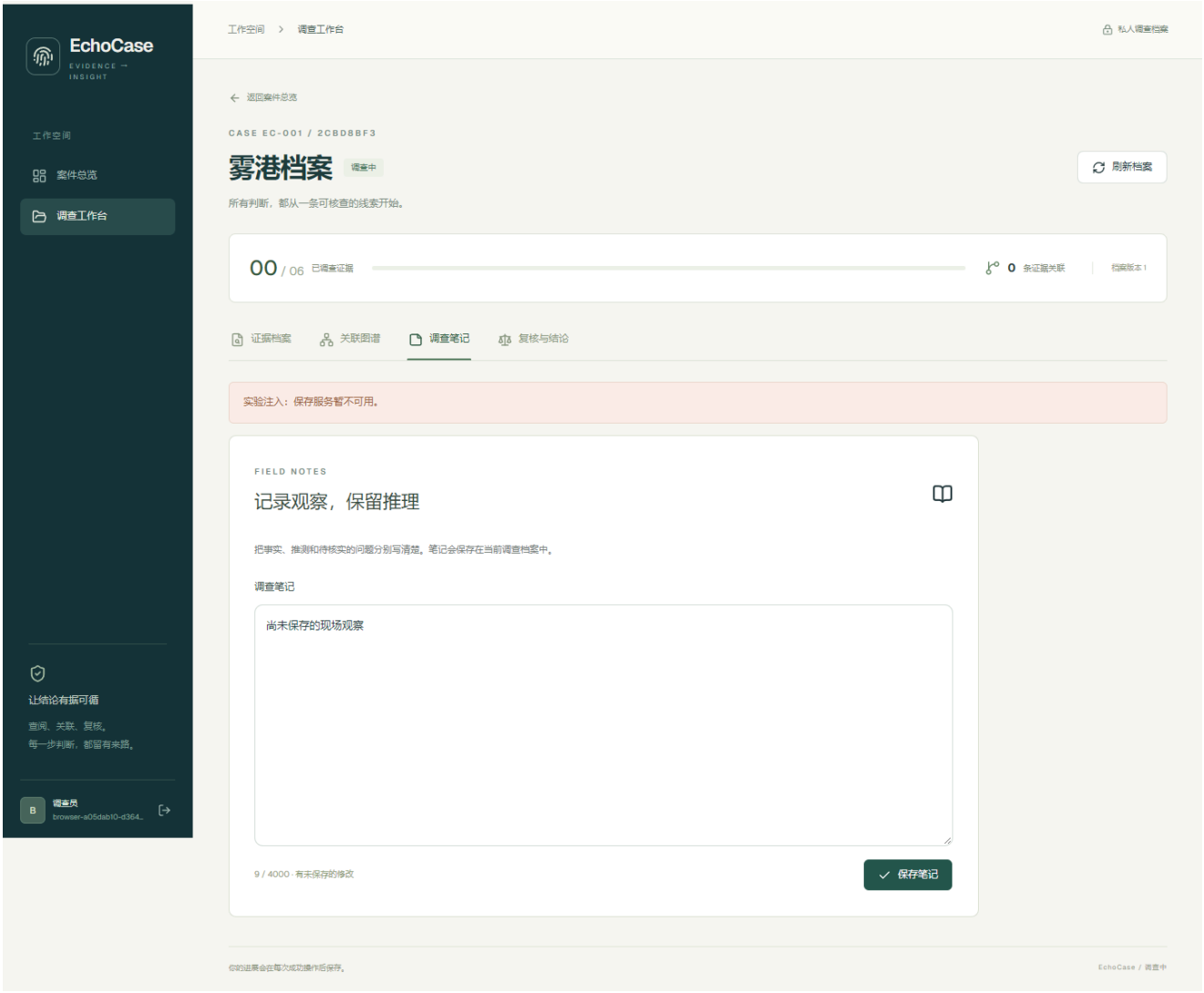


图 12-2　注入 503 后的笔记编辑器：错误可见，未保存草稿保留。该故障来自浏览器拦截，不是实际数据库中断。

实验 E12：把乐观失败做成可复现操作

Playwright 拦截 notes 写请求，返回带标记的 503。输入新草稿并保存，预期出现注入提示，编辑器仍保持草稿。随后绕过页面缓存直接请求服务器，预期数据库笔记仍为空。解除拦截，再次保存，预期成功。这项实验同时检验 UI、缓存和服务器事实，避免只看输入框还在就误判回滚成功。

双窗口测试则不用伪造 412：两个真实页面共享登录上下文，读取同一调查，A 成功保存后 B 用旧版本写入。断言 B 收到冲突且保留草稿，服务器仍为 A 内容。它补上了浏览器缓存与真实数据库并发的联动证据。

练习与参考答案

12-A： 为什么 dirty 草稿不能随着每次 refetch 自动更新？参考答案：草稿属于用户尚未提交的意图，后台快照刷新不应静默删除它。只有保存成功、明确放弃或有清晰合并流程时才替换。

12-B： invalidateQueries 是否立即保证页面读到最新？参考答案：它标记失效并触发符合条件的读取，仍存在网络、调度和并发时序。正确性不能依赖它即时完成；关键写入仍需版本条件。

12-C： 如何避免旧幂等回执覆盖更晚版本的缓存？参考答案：比较响应与缓存版本，较旧响应只用于确认原操作完成，再读取最新状态。当前简单顺序下通过重新失效读取恢复，复杂并发场景应增加显式版本单调性保护。

第 13 章 React Flow 与证据图谱的语义

图谱首先是业务模型的投影

节点表示已调查证据，边表示由调查员建立的相互印证或矛盾关系。未调查正文不通过图谱泄漏；尚未发现的证据不被绘制成可连接节点。React Flow 提供视口、节点拖动、端点和边的交互，但不负责判断一条关系是否具有业务合法性。

建立关联可通过拖动端点，也可通过选择起点、终点和类型后点击按钮。两种交互调用相同 API，用相同版本和幂等策略提交。键盘替代路径同时提升可访问性和自动化稳定性；E2E 不必依赖像素拖拽来覆盖业务正确性。

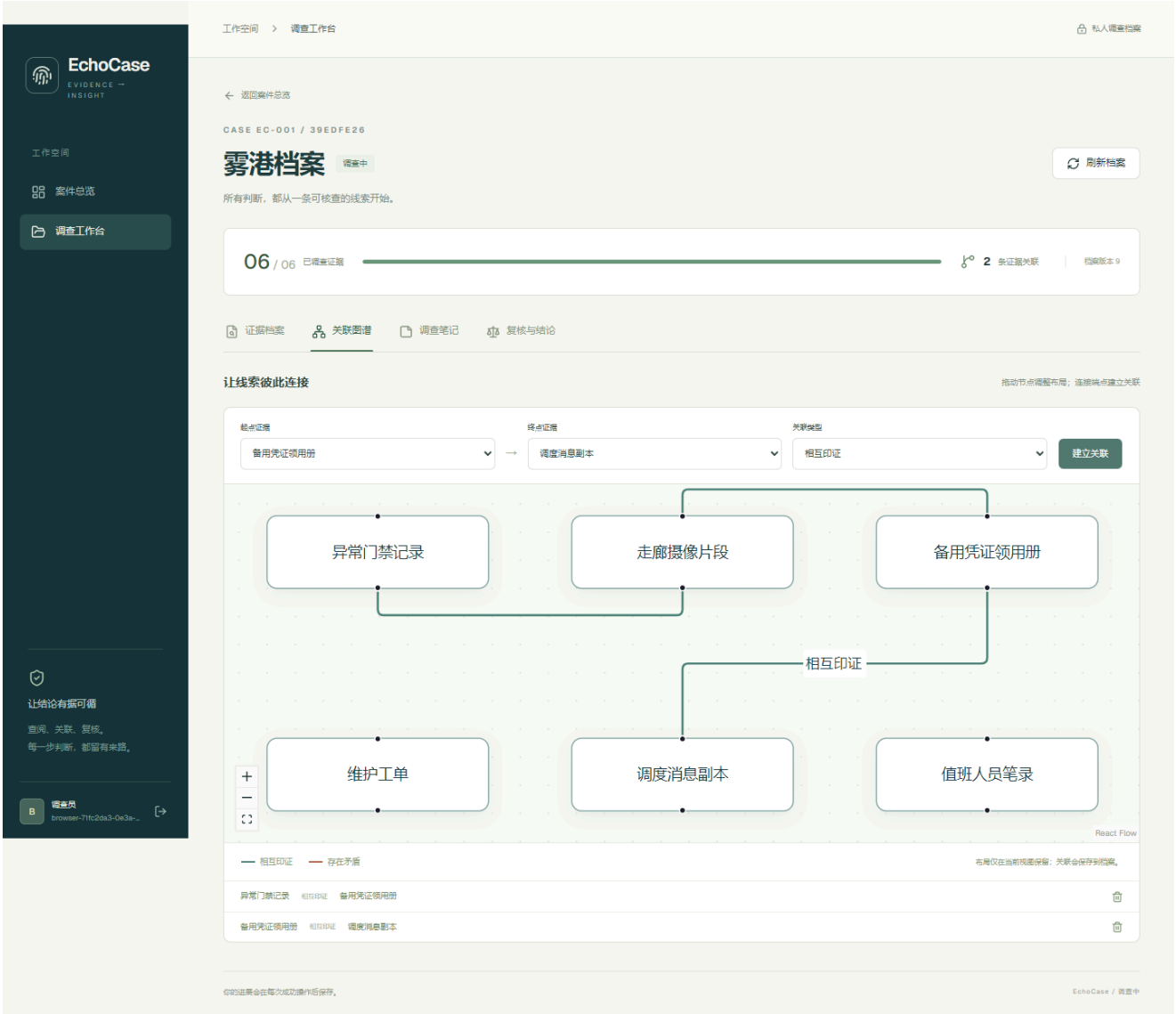


图 13-1　真实调查的证据图谱。节点来自已调查证据，关系保存到数据库，布局保留在当前视图。

本地节点位置与服务端关系分开

节点位置是呈现偏好，关联是业务事实。拖动一个节点不增加调查版本，也不会与另一窗口保存笔记发生冲突；建立或删除关联会推进根版本，并影响进入复核的门槛。这种划分让用户操作成本与业务含义一致。

React Flow 的受控节点使用 applyNodeChanges 更新组件状态。服务器快照变化后，已有节点保留原位置，新节点获得默认位置。离开视图后布局可以重建，界面对此有明确说明。若未来需要跨设备共享布局，应另建偏好模型或布局版本，避免把个人拖动当成调查证据变化。

无向边不能被控件字段误导

React Flow 连接数据使用 source 与 target，但领域关系是无向的。后端规范化两端顺序，去重 A–B 与 B–A；前端使用没有箭头的连线。颜色和标签共同表达关系类型，不只依赖颜色。否则色觉差异或黑白打印会失去含义。

删除关联通过边选择或文本关联列表完成。文本列表提供精确、可访问的删除按钮，不要求用户准确点击细线。删除也需要版本条件，提交后只读状态不能继续操作。一个“图形编辑器”仍然必须遵守整个调查状态机。

交互限制与服务端规则的重复

前端禁止自连，选择列表只包含已调查证据，非 Active 状态禁用连接。服务端仍重复检查这些规则，因为浏览器可被绕过，也可能在另一个窗口转换状态。对重复关联，前端没有用本地检查替代服务器唯一约束；服务器拒绝后显示明确错误并刷新。

图谱是可解释性工具，不是自动推理引擎。一条“相互印证”边代表用户的分析，不意味着服务器证明了两个陈述逻辑等价。评分只看固定案件规则与支持证据，不把图中边数当作现实推理可信度。

性能从规模假设开始

当前六个节点不需要虚拟化或复杂图布局服务。过早引入图数据库或分布式布局，会增加理解成本而没有可测收益。若节点增到数百，先测重渲染次数、拖动帧率、边路径计算和可见范围，再考虑 memo 化节点、分层展开与后台布局。

React Flow 包在前端主资源中占有明显体积，当前生产构建仍是一个主 JS 块。可以在后续把图谱视图懒加载，再对冷启动和切换耗时做对照。这个优化没有执行，因此本书不声称已经获得首屏性能改善。

实验 E13：同一关联能否被两种交互一致表达

先通过选择框建立门禁与凭证册的关联，确认图与文本列表都出现。刷新页面，关系仍存在；尝试反向重复建立，预期领域拒绝。删除后使用新的请求重新建立，确认版本与关联门槛变化。图谱截图来自真实页面，不是设计稿。

已有浏览器工作流验证了建立关联、图展示和最终提交；删除、自连和重复规则主要由领域与数据库测试覆盖。拖拽端点的精细手势、屏幕阅读器操作和大图性能没有独立实测，不能用已有选择框测试替代它们。

练习与参考答案

13-A： 为什么“保存布局”不应使用当前调查 Version？参考答案：布局不影响证据与结论语义。共用版本会产生无意义冲突；但若布局本身承载业务含义，如时间线排序，就必须重新分类并定义一致性规则。

13-B： 添加“因果关系”类型需要改变什么？参考答案：首先明确方向性，修改领域去重键和规范化逻辑，再更新数据库约束、API 合同、图箭头与测试。不能只增加一种颜色。

13-C： 设计一组图谱可访问性验收。参考答案：键盘选择两端、读取关系类型、建立与删除、明确焦点位置、无颜色依赖、错误提示可被辅助技术感知。当前只覆盖了其中部分能力。

第 14 章 单元、集成、架构与端到端测试

测试分层是为了解释失败

领域测试快速回答状态机和不变量是否正确；集成测试回答 EF 映射、数据库约束、身份与 HTTP 组合是否正确；浏览器测试回答 Cookie、表单、缓存、DOM 和真实网络组合是否正确。一个巨大的 E2E 套件可以发现问题，却未必能快速定位；只做单元测试则会漏掉本项目已经遇到的两个 ORM 与缓存缺陷。

层级	真实组件	被替代部分	主要证据
领域 NUnit	领域对象与规则	时间使用固定值	不变量、状态转换、失败不变性
架构 NUnit	编译后的程序集	无数据库	向内依赖
API 集成	ASP.NET 管道、Identity、EF、PostgreSQL	TestServer HTTP 宿主	事务、权限、并发、幂等
Playwright	Edge、HTTP、API、数据库	个别显式注入响应	完整流程、回滚、双窗口、移动端
SQL 实验	PostgreSQL 与连接	业务数据为合成	隔离与计划机制

当前最终后端结果为 38 / 38，浏览器结果为 4 组通过。没有用跳过集成测试的方式把缺数据库环境显示为绿色；测试连接缺失会明确失败。

测试数据隔离与清理

每次集成运行创建随机命名的 echocase_test_ 数据库，迁移到最终模式，建立两个账户，在结束时只删除该精确数据库。测试方法各自开始新的调查，不依赖前一方法留下的状态。共享登录客户端减少速率限制干扰，但不共享被断言的调查对象。

清理代码必须像实现一样被审查。数据库名称来自受控前缀与随机十六进制，使用引号保护；不会接受任意用户输入作为 DROP 目标。外部实验只停止本项目自己的容器，不重启整个 WSL，也不清空开发卷。失败也进入 finally 恢复，防止把故障留给用户。

并发测试需要独立上下文

两个任务共享一个 DbContext 并不是真实并发测试，因为 DbContext 不支持这样的线程使用；它可能先抛框架错误，掩盖业务竞争。当前 API 并发请求各有作用域，EF 双写测试明确创建两个上下文，SQL 隔离实验创建两个连接。

并发断言应检查整体结果：一方成功、一方冲突，数据库只保留一次更新，失败者没有回执。只断言“没有异常”或“最终版本大于 1”，无法识别双重提交。对同键重试，则应要求双方看到同一次操作结果，并确认审计只写一次。

浏览器测试验证用户可见结果

完整工作流从注册开始，调查六条证据，建立两条关联，保存笔记，刷新验证，进入复核，提交并刷新归档结果。测试使用角色和标签定位，不依赖 CSS 类序号来操作按钮；截图则记录关键稳定状态，作为教材插图与视觉审查材料。

回滚测试在浏览器层明确注入 503，再直接读取 API 核对服务器事实。双窗口冲突使用真实竞争，不伪造错误响应。移动端验证特定视口无横向溢出。测试没有把模拟响应标成真实数据库故障，真实停库由独立可靠性实验负责。

历史失败比“最后全绿”更有解释力

首次迁移缺少工具引用，导致集成库没有表；后续测试发现关联主键状态推断错误、审计唯一约束冲突映射错误；首轮浏览器发现清空 QueryCache 后登录观察器失联；可靠性实验发现 EF 包装异常没有映射为 503。这些失败保留了原始结果与修复后的证据。

它们支持的是“这些具体测试发现并阻止了这些缺陷”，而不是“测试可以证明没有缺陷”。当前没有统计代码覆盖率，没有运行长期压力、全面渗透、所有浏览器兼容或无障碍认证。验收报告应保留这些限制。

实验 E14：测试门禁是否真的会失败

对 QA 工具创建临时损坏副本：删除证据文件、改坏摘要、把预期通过状态改成不存在的枚举、引入外向依赖、让性能样本超过门槛。运行同一个检查入口，必须返回非零。再验证正常副本通过。这个自测保护的是验收工具本身，避免“发现错误但退出 0”的假门禁。

一个门禁测试只检查自己输出含 FAIL 仍然不足，因为 CI 以进程退出码决定后续步骤。需要捕获真实 returncode，并在自测中断言它。故意损坏的文件不提交进正常源目录，只把损坏类型与结果写入证据。

练习与参考答案

14-A： 哪个测试能发现登录页缓存问题？参考答案：浏览器或针对 QueryObserver 生命周期的前端集成测试。后端登录 API 测试无法观察 React 订阅关系。

14-B： 为什么不把所有测试都改用内存数据库？参考答案：它不能复现 PostgreSQL 外键、锁、隔离、SQL 状态与查询计划，也可能掩盖 EF 翻译差异。领域测试本来就不需要数据库；持久化测试应使用真实目标引擎。

14-C： 测试运行失败后，什么证据能区分代码缺陷与环境故障？参考答案：保存退出码、测试名称、错误形状、数据库版本、迁移状态和重现条件；修复环境后重新运行相关检查。不要直接把所有失败标记为 flaky。

第 15 章 SQL 执行计划、测量与性能回归

先问查询需要什么，再问索引建在哪里

个人调查列表按 OwnerId 过滤，再按 UpdatedAt 倒序取最近 50 条。只有 OwnerId 的索引能帮助过滤，却未必避免排序；只有 UpdatedAt 的索引可能必须跳过大量不属于该用户的行。组合索引把相等过滤与所需排序放在同一个访问路径中，正好对应当前查询形状。

性能实验建立十万条合成时间线，分布到五千个所有者，每行带固定长度负载。查询一个所有者，匹配二十行，按更新时间倒序取最多五十行。先无组合索引测量，再创建 `(owner_id, updated_at desc)`，执行 ANALYZE 后用相同 SQL 复测。数据规模与选择率公开，避免把不可比较的两次查询拼成优化故事。

如何读 EXPLAIN ANALYZE

EXPLAIN 给出计划器估计，ANALYZE 真正执行语句并记录实际观察。对写语句使用 EXPLAIN ANALYZE 会产生写入，不能在生产库随意运行。这里是专用合成只读查询，脚本保存 JSON 计划，便于程序检查 Seq Scan 与 Index 路径。

计划中的 cost 不是毫秒；actual time 是节点执行时间范围；rows 与 loops 需要结合读取；Buffers 帮助区分缓存命中与读取。一次看到 Index Scan 不代表总是更快：当结果覆盖大部分表时，顺序扫描可能更合理。索引还增加写放大、存储和维护成本。

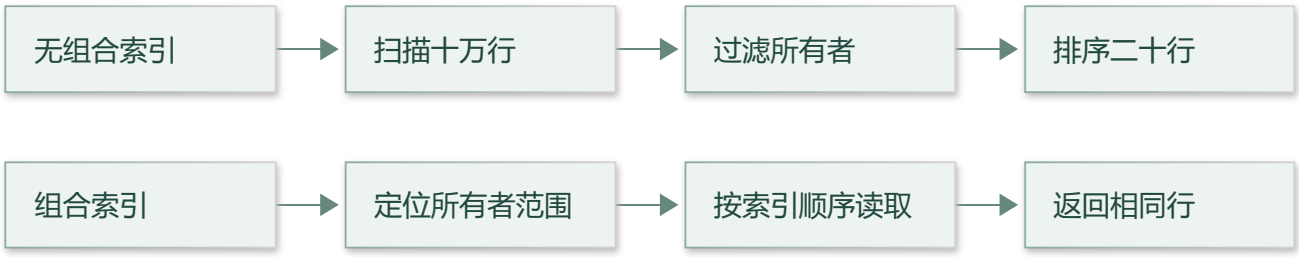


图 15-1　优化减少的是不必要的访问与排序，不是通过改变业务结果获取表面速度。

测量设计与实际结果

每个条件先预热三次，再记录二十一次端到端查询样本。中位数对少量离群值较稳健，p95 表达尾部；当前样本不足以做严格容量推断。网络、WSL、同时运行的构建与页面渲染都会影响端到端数字，因此同时保留数据库计划与所有样本。

条件	返回行数	端到端中位数
无组合索引	20	14.60 ms
有组合索引	20	1.18 ms

这组数字来自当前笔记本上的暖缓存合成表。它支持“在本次选择率与排序下，组合索引改变访问路径并降低测得延迟”，不支持“应用生产速度提升固定倍数”。真实应用增加相应迁移，并由数据保留测试验证升级，但没有用合成表的结果冒充完整 API 的优化对照。

API 测量是另一层证据

可靠性脚本对真实 HTTP 调查读取先预热五次，再顺序请求一百次，并验证每次状态为 200、追踪上下文可传播。当前 p95 为 26.10 ms。并发度为 1，因此这个值反映本机单请求路径的观察，不是最大吞吐、长时间稳定性或多用户尾延迟。

若要做负载测试，需要先定义到达率、并发模型、用户数据分布、持续时间、饱和指标和成功条件。闭环客户端会随着服务变慢而降低发压速率，可能隐藏真实排队；开环模型则要注意客户端自身是否成为瓶颈。本项目没有把短时串行测量称为企业级压测。

性能回归门禁如何避免噪声

当前 QA 对记录样本使用宽松的教学上限，并用故意超限输入证明检查确实失败。它证明门禁能识别被声明的性能条件，不代表这个阈值适合所有机器。CI 执行正确性测试与构建，保留本机性能证据；没有把不同硬件上的 p95 直接做严格同比。

更成熟的回归方案可以在受控硬件上比较同一数据集与同一负载，报告置信区间，并按关键路径设置预算。优化应同时观察 CPU、分配、数据库等待和错误率；单纯降低平均响应时间，可能把代价转移到失败率或内存。

实验 E15：计划变化能否解释时间变化

执行 database.py，保存前后 JSON 计划和样本。先只看计划预测哪种工作减少，再看测量是否一致。若时间没有改善，检查选择率、缓存、网络与采样噪声，不要强行改写结论。脚本要求两次返回行数一致，优化后计划确实使用索引路径，并在 finally 删除专用数据库。

进一步练习可以改变所有者数量，观察低选择率与高选择率时计划选择；这属于读者扩展实验，当前证据只覆盖已记录的五千所有者条件。不要把尚未执行的曲线画成测量结果。

练习与参考答案

15-A： 为什么组合索引顺序不能随便反过来？参考答案：前导列决定可以有效缩小的范围，等值 OwnerId 后跟排序时间，适合当前个人时间线。反向排列可能更适合全站时间流，但不是当前查询。

15-B： 加索引后写入变慢是否意味着优化失败？参考答案：需要结合读写比例与目标场景。索引维护是实际代价，若读延迟改善换来不可接受的写入或存储成本，应重新评估。

15-C： p95 26 ms 是否意味着 95% 的生产请求低于 26 ms？参考答案：不是。它只描述这次单机、暖机、串行、百次读取样本的经验分位，不能外推到其他负载和环境。

第 16 章 可观测性、依赖故障与恢复

可观测性从可回答的问题开始

当用户说“刚才保存失败”，需要区分未登录、CSRF、版本冲突、领域拒绝、数据库中断与未知程序错误。日志记录方法、路径、状态、耗时与 trace ID，Problem Details 把同一追踪编号返回给用户。这样可以从一次可见失败找到服务端记录，而不要求用户提供密码或完整请求体。

应用使用 ActivitySource 标记调查写操作，Meter 声明冲突与幂等重放计数器。当前没有部署完整 OpenTelemetry collector、指标看板或告警平台，因此只能声称代码具有这些仪表点和日志关联能力，不能声称已建立生产监控闭环。traceparent 传播则由真实 HTTP 实验验证。

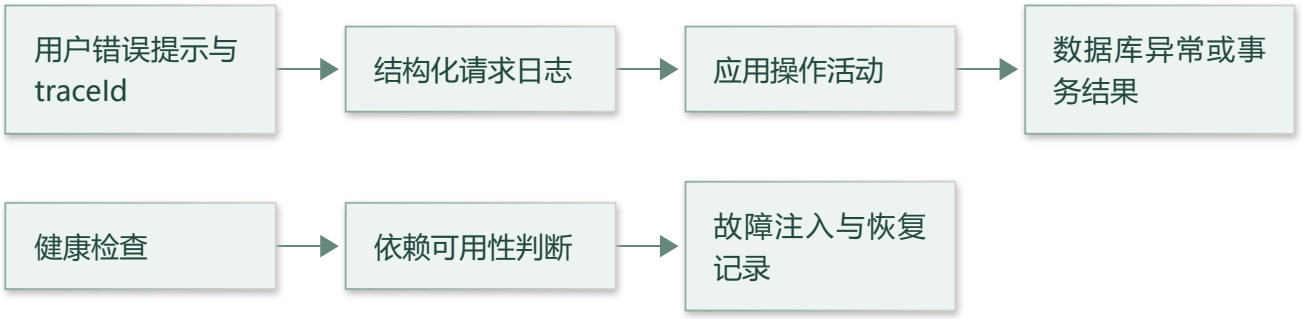


图 16-1　当前可观测链路。长期存储、告警路由和多实例聚合仍属于后续设计。

存活与准备状态必须分离

`/health/live` 只证明进程能响应；`/health/ready` 实际连接数据库。停库后，进程仍应存活，准备检查应失败，业务读取应返回 503。若把“数据库不可用”误判为“进程需要重启”，重启风暴可能增加恢复压力。

当前准备检查验证连接，不保证迁移版本完全正确，也不检查所有业务表可用性。部署流程先执行迁移再启动应用，弥补其中一部分风险；生产仍可以增加轻量模式检查或启动门禁。健康检查应快速、有界且不制造昂贵负载。

真正停止依赖，与浏览器返回假错误不同

浏览器回滚实验通过请求拦截返回 503，适合验证 UI 恢复；真实可靠性实验停止本项目 PostgreSQL 容器，验证连接池、异常包装、健康检查和恢复路径。前者无法证明数据库驱动错误处理，后者也不能单独证明编辑器保留草稿。两个实验覆盖不同边界。

脚本用 finally 启动被停止的容器，并轮询准备状态。它不停止其他容器，不删除数据库卷。故障窗口有界；如果恢复检查失败，脚本非零退出并留下明确错误。故障注入的可恢复性本身也属于实验设计。

实测结果与时间口径

观察	实际结果
数据库暂停后的 live	200
数据库暂停后的 ready	503
读取已有调查	503
启动命令返回后准备检查恢复	约 79 ms
恢复后原调查	可读取，ID 与保存前一致

“启动命令返回后”的计时不包含完整容器启动命令耗时，也不包含故障开始到检测的全部时间，因此不能当作系统 RTO。实验也没有执行跨主机备份恢复，不能给出生产 RPO。对时间口径的诚实解释，比一个很小的恢复数字更重要。

首次断库返回 500 的原因见第 4 章：EF 包装底层连接异常，原匹配漏掉它。修正后用相同故障再运行，结果符合上表。故障恢复后读取原调查，证明既有提交未因为进程重连而消失；它不证明故障期间的未知写结果，应另以幂等键确认。

重试预算与故障放大

没有引入自动无限重试。数据库连接池会重建失效连接，但业务写入仍遵循显式幂等协议。若在每层都配置三次重试，一个请求可能放大成多倍数据库访问；在依赖最脆弱时制造更大压力。生产方案应设定总超时、重试次数、退避、抖动和可重试条件，并观察真实恢复效果。

当前限流是进程内分区，在多实例中不会形成共享总额度。当前日志是本地输出，没有不可篡改审计存储。当前审计与业务同库，能支持操作追踪，却不能防止拥有数据库写权限者修改历史。这些限制不会因“observability”标签而消失。

实验 E16：从症状到机制的排障

执行 reliability.py 前确认开发 API 可用、没有其他实验正在使用数据库。脚本先采集正常读取样本，再暂停依赖，核对三个状态，最后恢复并读取原调查。保存 JSON 结果、失败前日志和修复说明，按“症状→证据→假设→修正→重复实验”写一页复盘。

不要因为最终准备状态恢复，就删除失败记录。失败记录解释了错误映射为何需要改变，也提供未来回归时的已知症状。清理指的是恢复故障环境，不是抹去工程历史。

练习与参考答案

16-A： 数据库已经恢复，为什么还有短暂 503？参考答案：连接池旧连接、启动恢复、DNS/网络、事务取消与应用重试节奏都可能影响观察。应检查具体异常与时间线，不直接增加无限重试。

16-B： trace ID 为什么不能包含邮箱？参考答案：追踪编号应作为非语义关联标识，避免把个人信息扩散到日志、代理和监控。身份可以通过受控字段另行关联，并遵守保留策略。

16-C： 设计真正 RTO/RPO 实验需要什么？参考答案：定义故障起点、检测与恢复终点，持续记录已确认写入序列，使用独立备份介质或复制目标恢复，再比较恢复数据。当前容器重启实验不能替代这些步骤。

第 17 章 容器、CI 与 Azure 部署边界

一份可运行镜像包含哪些东西

多阶段 Dockerfile 先用固定 Node 镜像执行 npm ci 与前端构建，再用固定 .NET SDK 镜像按锁文件还原与发布，最终只保留 ASP.NET Runtime、发布输出和前端静态资源。运行阶段使用镜像提供的非 root UID，监听 8080。前端与 API 同源，避免为部署方便引入宽松 CORS。

依赖锁文件保证包解析结果，固定基础镜像标签缩小版本漂移，但标签仍可能重新指向不同镜像。完整供应链固定需要解析并维护 digest，结合漏洞扫描、更新与来源证明。本项目记录实际构建 image ID，没有声称已经建立签名、SBOM 或位级可复现构建。

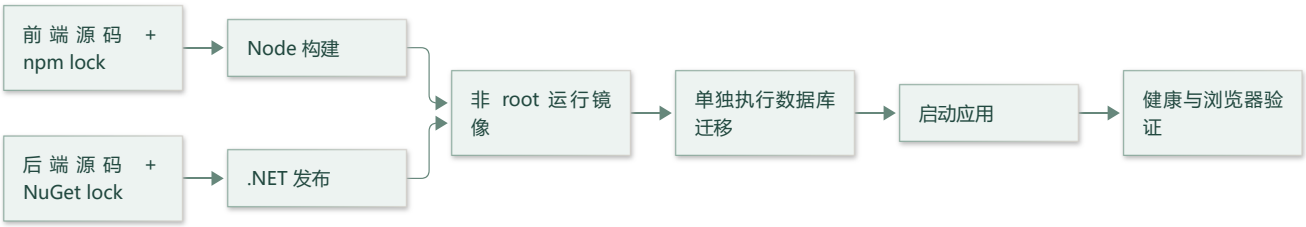


图 17-1　迁移是显式步骤，不让每个应用副本在启动时同时争抢模式升级。

重建与持久状态

应用容器应可删除重建，数据库卷与 Data Protection 密钥卷则独立保留。容器实验使用独立 PostgreSQL 实例，从空模式运行迁移，注册账户、创建调查，再删除并重建应用容器。随后使用原 Cookie 读取原调查，检验数据库与密钥的持久边界。

这里验证的是同机容器重建，不是异机灾难恢复。数据库卷仍与机器处于同一故障域；密钥卷没有加密备份或轮换演练。部署说明明确保留卷的停止命令，删除卷需要由所有者确认数据不再需要。

CI 应执行什么

GitHub Actions 使用真实 PostgreSQL service，运行锁定还原、后端测试、前端生成合同、类型与构建、浏览器测试和证据门禁。构建门禁不应通过路径过滤悄悄跳过关键检查；失败后后续依赖步骤应停止，测试和日志尽可能上传为制品。

本机执行与远程 Actions 是两种证据。验收报告记录实际远程运行状态；工作流文件存在不算已验证。若远程基础设施或权限导致阻塞，应保留运行链接和失败原因，不把本地全绿改写成远程成功。

本地重建实验的限制

Podman 初始开发数据库采用 pasta 网络，不能直接接入桥接网络。首次容器实验暴露了脚本对 inspect 输出的假设。修正创建独立桥接网络和独立容器数据库，避免依赖已经配置好的开发模式。这比临时改动共享容器网络更利于验证干净环境。

本地 Compose 描述提供同样的服务关系：数据库就绪、迁移完成、应用启动。实际验收分别标注命令路径；直接 Podman 验证不能自动证明任意版本 Compose provider 都兼容。平台差异应记录，而不是把“都是 OCI”当成所有编排行为一致的证据。

Azure 是部署设计，不是本机验证的延伸

可选择 Azure App Service 运行一个容器，配 PostgreSQL Flexible Server。部署前需要确定订阅预算、区域、网络连接、HTTPS、AllowedHosts、迁移身份、运行身份、密钥保护、日志去向和备份恢复。官方自定义容器文档说明端口和镜像配置；托管身份可用于适当的资源访问，但仍需给数据库身份授予精确权限。

本项目不执行资源创建、不设置付费套餐、不产生云费用。Azure 的网络、托管身份、生产 Cookie、伸缩与备份恢复均为 NOT_RUN。实际部署应由所有者在审阅具体资源清单与费用后批准，而不是运行一个带默认付费选项的“一键上线”脚本。

关注点	本地已验证	Azure 仍需验证
进程与静态资源	OCI 容器运行	平台端口、启动与探针配置
数据库	PostgreSQL 17.7	网络、TLS、身份、容量与备份
Cookie	本地 HTTP 与密钥保留	HTTPS、代理头、持久密钥保护
迁移	新建与增量保留	部署窗口、权限分离、回退策略
运行观测	本地日志与 trace ID	集中日志、告警与运维响应

实验 E17：删除可丢弃部分，验证持久部分

运行 container.py。要求镜像成功构建、应用使用非 root 用户、静态页面可读取、数据库迁移成功、原账户 Cookie 与调查在应用重建后仍可用。检查结构化结果而非仅查看容器状态为 Up。容器运行不代表业务已准备好，必须访问健康与业务端点。

实验结束保留本地应用供审阅；需要停止时只停止本项目的 app 与数据库容器，保留卷。若要删除镜像或卷，应先明确是否仍需复现实验或保留调查数据。

练习与参考答案

17-A： 为什么迁移不在每个应用实例启动时自动执行？参考答案：会把运行流量与模式变化耦合，多个副本可能竞争，失败责任也模糊。单独步骤便于权限分离、审批与回退设计。

17-B： 锁文件是否证明供应链安全？参考答案：它固定解析结果，不保证包无漏洞或来源可信。仍需扫描、更新、校验来源和合适的制品证明。

17-C： 准备 Azure 审批时应交付什么？参考答案：具体资源、费用估算、网络与身份设计、迁移和恢复步骤、验证计划及清理方案。当前只有设计材料，没有创建任何资源。

第 18 章 综合考核、证据审查与后续工程

综合任务：解释一个操作的一生

选择“提交结论”作为考核场景。先从浏览器表单追踪请求正文、Cookie、防伪头、幂等键和 If-Match；再找到 API 绑定、应用指纹、资源加载、领域 Submit、EF 状态变化和数据库事务。最后从新页面读取封存结果，说明哪一步证明持久化、哪一步只是内存状态。

合格答案不应只画四层框。需要解释复核状态为什么被冻结，支持证据为什么必须来自已发现集合，重复请求为什么不会重复提交，另一个账户为什么不能重放回执，数据库断开时客户端如何判断结果未知。能把这些边界连起来，才体现完整的软件工程能力。

考核一：受控故障排查

给考生三种症状，要求先收集证据再修正：登录 API 返回 200 但页面仍停留；没有其他用户却出现 EF 并发异常；数据库暂停时业务返回 500。对应本项目真实发现的缓存观察器、主键生成映射和包装异常问题。

评分不以“猜中某个关键词”为准，而是检查能否提出可证伪假设、选取最小实验、解释反例并在修复后重复相同场景。修正后只跑一次不相关单元测试，不足以证明缺陷已闭合；修改错误映射后应再次真实断库。

能力	观察行为	不充分的替代物
领域建模	能用状态机解释非法动作	只列实体名称
一致性	能说明数据库原子边界	只说使用了事务
全栈恢复	保留草稿并处理旧版本	自动覆盖冲突
安全	从可信身份过滤资源	隐藏按钮或随机 ID
性能	控制变量并读执行计划	单次秒表或固定倍数口号
可靠性	区分注入、观测与恢复口径	容器 Up 截图

考核二：维护性变更分析

提出需求：“结论可以被撤回一次，撤回后允许继续调查。”先列语义问题：谁能撤回、何时允许、原分数与支持证据是否保留、审计如何表达、撤回后幂等重放应返回什么、其他窗口看到封存版本时如何恢复。然后才讨论枚举和端点。

当前实现没有该功能。考生应提交设计与测试计划，或在独立分支完整实现并提供新证据；不能直接删除只读检查。一个看似小的状态边会同时影响领域、数据库 CHECK、API、前端交互与教材说明，是维护性评审的理想练习。

考核三：审查一个“全绿”报告

审阅者拿到 TRX、Playwright JSON、SQL 计划、恢复结果、容器结果、PDF 和验收报告。首先核对它们是否来自同一代码与版本，再检查是否存在跳过测试、空集合通过、坏输入返回零、缺失文件、无来源数字或未标记的模拟。

自动门禁能检查文件存在、摘要、结构化结果、来源绑定的数值和章节代码片段边界。它无法理解所有自然语言推论，因此还需要逐条审查重要结论。例如“启动后 79 ms 可读”与“RTO 为 79 ms”只差几个字，证据支持程度却不同。

发布验收如何分层

工程验收检查应用、数据库、浏览器、容器和实验；出版验收检查目录、链接、图意、代码、可读性与逐页视觉；外部审阅检查教学是否适合目标读者。三者没有一个能自动代替其他两者。

本书是 review edition。页数按实质内容和版面生成，不通过重复日志、空白页或放大字体凑成目标。验收报告给出实际页数与原定 300–400 页目标的差异，并标明后续编辑、外部读者反馈和生产验证事项。若某个目标未完成，就保留 PARTIAL，不把它包装成已满足全部最终条件。

继续深化的优先级

优先补齐能改变当前使用风险的工作：账户恢复与会话撤销、复杂冲突差异界面、证据目录版本、查询分页、回执保留、备份恢复与指标导出。随后基于实测决定是否懒加载图谱、是否使用查询投影、是否需要更细功能模块。

仍然不需要为了简历标签增加微服务、Kubernetes、AI 代理或多租户。每一项新技术应对应一个不能由现有方案合理解决的需求，并能用实验解释收益与代价。工程深度来自理解边界和失败，不来自部署组件数量。

练习与参考答案

18-A： 给本项目最强的一项证据与最弱的一项承诺各举一例。参考答案：真实双窗口冲突加数据库状态核对是较强证据；“生产就绪”没有长期运行、账户治理、灾难恢复与云环境证据，不能作为已实现承诺。

18-B： 如何让独立验收更独立？参考答案：由未参与实现的人在干净环境按文档复现，使用独立验收问题与故障注入，记录与作者解释不一致的结果。当前独立成册报告不等于第三方审计。

18-C： 在完成学习后写一页个人复盘，分别说明你亲自执行、阅读推导和仍未验证的内容。参考答案应有明确范围与来源，不把作者在本机执行的实验描述成自己的生产经验。

附录 A 实验复现手册

以下记录与仓库 docs/experiments 同源生成。每项包含问题、机制、基线、执行、观测、解释、限制和清理。状态来自真实结构化证据；无法确认的项目保留 NOT_RUN。

E01 依赖边界退化

状态：**VERIFIED**。来源：`src/EchoCase.Domain/EchoCase.Domain.csproj`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

Domain 引入外向依赖时，门禁能否阻止？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

项目引用是编译时知识；运行时调用方向不等于引用方向。

3. 基线与实验设计

依赖门禁检查真实程序集；故障自测只在临时副本注入 Infrastructure 引用。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~CoreHasNoFrameworkOrOutwardDependencies"
```

5. 实测观测

正常边界成立，坏输入必须非零退出。 对应测试 CoreHasNoFrameworkOrOutwardDependencies：Passed。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

项目引用是编译时知识；运行时调用方向不等于引用方向。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `src/EchoCase.Domain/EchoCase.Domain.csproj` 与本章问题具有直接关系。

7. 限制与替代解释

程序集检查不能识别所有职责混杂；还需人工审查变化原因。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E02 领域脱离框架

状态：**VERIFIED**。来源：`src/EchoCase.Domain/Investigation.cs`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

不启动数据库是否仍可验证领域与模板取舍？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

领域不依赖持久化或 HTTP，应用端口不返回 IQueryable。

3. 基线与实验设计

直接构造调查，用固定时间驱动业务方法，结合上游机制审计。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~StartRequiresOwner"
```

5. 实测观测

拒绝没有所有者的调查，领域测试不建立数据库连接。 对应测试 StartRequiresOwner：Passed。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

领域不依赖持久化或 HTTP，应用端口不返回 IQueryable。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `src/EchoCase.Domain/Investigation.cs` 与本章问题具有直接关系。

7. 限制与替代解释

这证明当前规则可独立验证，不证明所有未来变化都不涉及外层。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E03 解锁与状态机

状态：**VERIFIED**。来源：`tests/EchoCase.Tests/DomainTests.cs`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

非法操作是否既被拒绝又不推进版本？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

先验证再修改，终态只读，复核门槛依赖聚合集合。

3. 基线与实验设计

先尝试锁定证据，再按门禁→凭证册→调度顺序调查，验证解锁链。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~HiddenEvidenceCannotBeExamined"
```

5. 实测观测

锁定路径被拒绝，Version 保持 1；合法路径逐次推进。 对应测试 HiddenEvidenceCannotBeExamined：Passed。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

先验证再修改，终态只读，复核门槛依赖聚合集合。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `tests/EchoCase.Tests/DomainTests.cs` 与本章问题具有直接关系。

7. 限制与替代解释

仅覆盖固定案件；动态案件规则版本尚未实现。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E04 请求管道保护

状态：**VERIFIED**。来源：`src/EchoCase.Api/Program.cs`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

认证 Cookie 是否足以让缺防伪头的写请求成功？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

Cookie 自动发送，CSRF 令牌提供独立请求证明。

3. 基线与实验设计

使用已登录客户端省略 X-CSRF-TOKEN，保持其他请求条件可观察。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~CsrfIsRequiredEvenWithValidCookie"
```

5. 实测观测

缺防伪头返回 400，受保护写入未完成。 对应测试 CsrfIsRequiredEvenWithValidCookie：Passed。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

Cookie 自动发送，CSRF 令牌提供独立请求证明。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `src/EchoCase.Api/Program.cs` 与本章问题具有直接关系。

7. 限制与替代解释

没有进行全面浏览器跨站攻击工具扫描；此处是明确负向合同。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E05 失败原子性

状态： **VERIFIED** 。 来源： `src/EchoCase.Infrastructure/PostgresInvestigationStore.cs` 。 证据： `evidence/backend/backend.trx`。

1. 工程问题与假设

领域拒绝后是否留下审计或幂等回执？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

业务状态、审计、回执共用事务；异常导致回滚。

3. 基线与实验设计

新调查直接请求进入复核，随后以新上下文查询回执和审计。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~DomainFailureRollsBackReceiptAndAudit"
```

5. 实测观测

422，失败键无回执，审计仍只有启动记录。 对应测试 DomainFailureRollsBackReceiptAndAudit：Passed。结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

业务状态、审计、回执共用事务；异常导致回滚。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `src/EchoCase.Infrastructure/PostgresInvestigationStore.cs` 与本章问题具有直接关系。

7. 限制与替代解释

未注入数据库 COMMIT 确认丢失；传输未知结果需另一类实验。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E06 合同污染

状态：**VERIFIED**。来源：`frontend/src/api/schema.d.ts`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

null 或伪造额外 OwnerId 是否被静默接受？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

严格构造参数、非空注解与未知成员拒绝。

3. 基线与实验设计

分别发送 notes:null 与多出的 ownerId 字段。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~MalformedAndNullContractsAreRejected"
```

5. 实测观测

两者均返回 400，客户端无法通过 JSON 指定所有者。 对应测试 MalformedAndNullContractsAreRejected：Passed。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

严格构造参数、非空注解与未知成员拒绝。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `frontend/src/api/schema.d.ts` 与本章问题具有直接关系。

7. 限制与替代解释

生成类型提供编译约束，不做运行时 JSON schema 验证。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E07 迁移与关系约束

状态：**VERIFIED**。来源：`src/EchoCase.Infrastructure/Migrations`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

增量迁移是否保留数据，直接坏写能否被数据库拒绝？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

迁移历史、组合索引、复合外键和 CHECK。

3. 基线与实验设计

专用库先应用初始迁移，写笔记，再升级；另一测试直插未知发现关联。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter
"FullyQualifiedName~UpgradePreservesExistingInvestigationAndEfRejectsStaleTrackedWriter"
```

5. 实测观测

数据保留且至少两个迁移；非法关联触发 23503。对应测试 UpgradePreservesExistingInvestigationAndEfRejectsStaleTrackedWriter：Passed。结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

迁移历史、组合索引、复合外键和 CHECK。因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `src/EchoCase.Infrastructure/Migrations` 与本章问题具有直接关系。

7. 限制与替代解释

真实云滚动升级、巨大表索引在线构建没有运行。单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E08 丢失更新与版本

状态：**VERIFIED**。来源：`tests/EchoCase.Tests/PostgresApiTests.cs`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

两个相同旧版本写请求是否恰有一胜？假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

SQL 条件更新与 EF 并发令牌保护聚合版本。

3. 基线与实验设计

不同键并发写笔记；另用两个独立 DbContext 保存相同旧版本。先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~DifferentKeysCompetingOnOneVersionHaveOneWinner"
```

5. 实测观测

API 状态为 200/412；EF 旧上下文保存抛并发异常。 对应测试 DifferentKeysCompetingOnOneVersionHaveOneWinner：Passed。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

SQL 条件更新与 EF 并发令牌保护聚合版本。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `tests/EchoCase.Tests/PostgresApiTests.cs` 与本章问题具有直接关系。

7. 限制与替代解释

这不是长期压力测试；受控交错有利于机制解释。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E09 重复请求去重

状态： **VERIFIED** 。 来源： `src/EchoCase.Infrastructure/PostgresInvestigationStore.cs` 。 证据： `evidence/backend/backend.trx`。

1. 工程问题与假设

相同键并发写入是否只提交一次？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

数据库事务级锁加持久回执，同键同指纹返回原快照。

3. 基线与实验设计

两个请求共享键、版本与正文，查询提交后的审计数量。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~SameKeyConcurrentRequestsCommitOnce"
```

5. 实测观测

两个响应版本相同，仅一条 notes 审计。 对应测试 SameKeyConcurrentRequestsCommitOnce：Passed。结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

数据库事务级锁加持久回执，同键同指纹返回原快照。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `src/EchoCase.Infrastructure/PostgresInvestigationStore.cs` 与本章问题具有直接关系。

7. 限制与替代解释

保证受回执保留期和所有写路径遵循端口约束。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E10 水平越权

状态：**VERIFIED**。来源：`tests/EchoCase.Tests/PostgresApiTests.cs`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

合法登录用户能否访问另一账户调查？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

可信 OwnerId 与资源 ID 联合过滤，拒绝存在性泄漏。

3. 基线与实验设计

Alice 创建调查；Bob 分别读取和修改该 ID。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~ForeignOwnerCannotReadOrWrite"
```

5. 实测观测

两种请求均为 404，原调查保持属于 Alice。 对应测试 ForeignOwnerCannotReadOrWrite：Passed。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

可信 OwnerId 与资源 ID 联合过滤，拒绝存在性泄漏。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `tests/EchoCase.Tests/PostgresApiTests.cs` 与本章问题具有直接关系。

7. 限制与替代解释

未完成 MFA、邮箱确认、安全戳会话撤销等生产身份能力。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E11 浏览器持久工作流

状态：**VERIFIED**。来源：`frontend/tests/workflow.spec.ts`。证据：`evidence/browser/results.json`。

1. 工程问题与假设

刷新后是否仍可完成五模块调查？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

真实 Cookie、HTTP、API、EF 与 PostgreSQL 全链路。

3. 基线与实验设计

注册、启动、调查、关联、笔记、刷新、复核、提交、再次刷新。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
cd frontend
node node_modules/@playwright/test/cli.js test
```

5. 实测观测

完整工作流通过，封存结果评分 100，截图由实际页面采集。 当前浏览器结果：4 通过、0 失败。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

真实 Cookie、HTTP、API、EF 与 PostgreSQL 全链路。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `frontend/tests/workflow.spec.ts` 与本章问题具有直接关系。

7. 限制与替代解释

单一虚构案件；不代表复杂故事或所有设备兼容。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E12 乐观回滚

状态：**VERIFIED**。来源：`frontend/src/App.tsx`。证据：`evidence/browser/results.json`。

1. 工程问题与假设

注入保存 503 后，草稿与服务端事实是否正确分离？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

onMutate 快照、onError 回滚、dirty 草稿独立于 Query 缓存。

3. 基线与实验设计

仅拦截 notes 响应，保存草稿后直接读取服务器；解除拦截重试。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
cd frontend
node node_modules/@playwright/test/cli.js test
```

5. 实测观测

错误可见，草稿保留，服务器仍为空；解除后保存成功。 当前浏览器结果：4 通过、0 失败。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

onMutate 快照、onError 回滚、dirty 草稿独立于 Query 缓存。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `frontend/src/App.tsx` 与本章问题具有直接关系。

7. 限制与替代解释

此故障为浏览器注入，不是真实数据库暂停。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E13 图谱关联

状态：**VERIFIED**。来源：`frontend/src/EvidenceGraph.tsx`。证据：`evidence/browser/results.json`。

1. 工程问题与假设

图与文本交互是否对应持久化关系？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

React Flow 投影与无向边规范化，服务端校验发现与唯一性。

3. 基线与实验设计

调查证据后以选择框建立两条关联，展示图谱并刷新后提交。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
cd frontend
node node_modules/@playwright/test/cli.js test
```

5. 实测观测

图谱与提交工作流通过；域测试覆盖自环、反向重复与未发现端点。 当前浏览器结果：4 通过、0 失败。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

React Flow 投影与无向边规范化，服务端校验发现与唯一性。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `frontend/src/EvidenceGraph.tsx` 与本章问题具有直接关系。

7. 限制与替代解释

端点拖拽手势、大图性能与屏幕阅读器尚未独立测试。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E14 验收门禁自测

状态：**VERIFIED**。来源：`tools/test_gate.py`。证据：`evidence/qa/gate-selftest.json`。

1. 工程问题与假设

检查器是否真的拒绝坏输入？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

退出码而非日志措辞控制门禁；临时副本隔离故障。

3. 基线与实验设计

删除文件、修改摘要、引入外向引用、伪造状态、超限性能、删答案和伪造测量。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python tools/test_gate.py
```

5. 实测观测

正常基线必须返回 0，七类损坏各自非零。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

退出码而非日志措辞控制门禁；临时副本隔离故障。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `tools/test_gate.py` 与本章问题具有直接关系。

7. 限制与替代解释

有限规则检查不能证明所有自然语言论断。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

临时副本与受控坏 PDF 由 TemporaryDirectory 自动清理，原始源和成品不被修改。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E15 查询计划优化

状态：**VERIFIED**。来源：`experiments/database.py`。证据：`evidence/database/results.json`。

1. 工程问题与假设

匹配过滤和排序的组合索引是否减少工作？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

B-tree 前导等值范围与有序访问，ANALYZE 更新统计。

3. 基线与实验设计

十万条、五千所有者；两条件各预热三次并测二十一次，保留 JSON 计划。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python experiments/database.py
```

5. 实测观测

返回行数均为 20，Seq Scan 与索引计划可对照，测量来自保存样本。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

B-tree 前导等值范围与有序访问，ANALYZE 更新统计。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `experiments/database.py` 与本章问题具有直接关系。

7. 限制与替代解释

合成单机暖缓存表；不是完整应用吞吐或生产容量。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

脚本 finally 删除本次随机 echocase_lab_ 数据库；开发数据库不被清空。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E16 真实依赖故障

状态：**VERIFIED**。来源：`experiments/reliability.py`。证据：`evidence/reliability/results.json`。

1. 工程问题与假设

数据库停用时能否正确降级并恢复原调查？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

连接池失效、异常包装、live/ready 分离与持久卷。

3. 基线与实验设计

真实停止 echocase-postgres，核对健康与业务状态，finally 恢复。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python experiments/reliability.py
```

5. 实测观测

200/503/503，恢复后原调查可读；首次 500 缺陷已修正。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

连接池失效、异常包装、live/ready 分离与持久卷。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `experiments/reliability.py` 与本章问题具有直接关系。

7. 限制与替代解释

恢复时间从启动命令返回后计，不等于端到端 RTO。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

finally 启动被停止的 echocase-postgres；脚本非零时先检查容器与 ready，保留失败证据。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E17 容器重建

状态：**VERIFIED**。来源：`experiments/container.py`。证据：`evidence/container/results.json`。

1. 工程问题与假设

删除应用容器后状态与 Cookie 是否仍成立？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

镜像无状态，数据库与 Data Protection 密钥卷持久化。

3. 基线与实验设计

独立数据库迁移、创建账户与调查、删除重建应用，再读取原调查。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python experiments/container.py
```

5. 实测观测

非 root 运行、静态资源服务、调查与 Cookie 均需存活。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

镜像无状态，数据库与 Data Protection 密钥卷持久化。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `experiments/container.py` 与本章问题具有直接关系。

7. 限制与替代解释

只验证本地 OCI 路径；Compose provider 与 Azure 未运行。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

审阅后停止 echocase-app 与 echocase-container-db，保留数据库与密钥卷。删除卷会丢失数据，非自动清理步骤。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E18 不可重复读

状态：**VERIFIED**。来源：`experiments/database.py`。证据：`evidence/database/results.json`。

1. 工程问题与假设

同一事务两次读取在不同隔离级别是否相同？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

READ COMMITTED 语句快照与 REPEATABLE READ 事务快照。

3. 基线与实验设计

A 读 10，B 写 20，再让 A 读取；分别使用两种隔离。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python experiments/database.py
```

5. 实测观测

READ COMMITTED 为 10/20；REPEATABLE READ 为 10/10。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

READ COMMITTED 语句快照与 REPEATABLE READ 事务快照。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `experiments/database.py` 与本章问题具有直接关系。

7. 限制与替代解释

受控两个连接，不代表所有隔离现象都已覆盖。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

脚本 finally 删除本次随机 echocase_lab_ 数据库；开发数据库不被清空。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E19 写偏斜与可串行化

状态：**VERIFIED**。来源：`experiments/database.py`。证据：`evidence/database/results.json`。

1. 工程问题与假设

快照隔离是否保证至少一人值班？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

跨行不变量与序列化依赖检测；40001 需要整体事务重试。

3. 基线与实验设计

两事务先读两人在岗，再分别离岗；比较 Repeatable Read 与 Serializable。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python experiments/database.py
```

5. 实测观测

前者可无人值班，后者第二写入 40001 并保留一人。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

跨行不变量与序列化依赖检测；40001 需要整体事务重试。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `experiments/database.py` 与本章问题具有直接关系。

7. 限制与替代解释

教学 on_call 表，不是 EchoCase 业务功能；用于解释聚合边界。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

脚本 finally 删除本次随机 echocase_lab_ 数据库；开发数据库不被清空。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E20 幂等键误复用

状态：**VERIFIED**。来源：`src/EchoCase.Application/Investigations.cs`。证据：`evidence/backend/backend.trx`。

1. 工程问题与假设

相同键改变正文是否被错误重放？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

请求指纹绑定用户意图，键不是任意缓存 ID。

3. 基线与实验设计

先保存 first，再以相同键保存 different。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
dotnet test --filter "FullyQualifiedName~ReusedKeyWithChangedPayloadIsConflict"
```

5. 实测观测

第二请求为 409，不能变成新写入或返回无关成功。 对应测试 ReusedKeyWithChangedPayloadIsConflict：Passed。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

请求指纹绑定用户意图，键不是任意缓存 ID。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `src/EchoCase.Application/Investigations.cs` 与本章问题具有直接关系。

7. 限制与替代解释

当前规范化基于强类型序列化；跨语言通用规范化未实现。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E21 追踪上下文

状态：**VERIFIED**。来源：`experiments/reliability.py`。证据：`evidence/reliability/results.json`。

1. 工程问题与假设

服务端是否沿用有效上游 traceparent？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

W3C Trace Context 与 ASP.NET 请求活动。

3. 基线与实验设计

真实 HTTP 读取带固定有效 traceparent，逐次检查 X-Trace-Id。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python experiments/reliability.py
```

5. 实测观测

一百次测量读取的追踪 ID 与输入一致。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

W3C Trace Context 与 ASP.NET 请求活动。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `experiments/reliability.py` 与本章问题具有直接关系。

7. 限制与替代解释

没有验证外部 collector、采样与跨服务导出。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

finally 启动被停止的 echocase-postgres；脚本非零时先检查容器与 ready，保留失败证据。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E22 性能回归拒绝

状态：**VERIFIED**。来源：`tools/test_gate.py`。证据：`evidence/qa/gate-selftest.json`。

1. 工程问题与假设

超出教学阈值的样本是否使门禁失败？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

结构化样本阈值与语义检查，避免仅摘要失败掩盖真实检查。

3. 基线与实验设计

在临时副本把 p95 改为 9999，并刷新摘要，单独触发性能条件。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python tools/test_gate.py
```

5. 实测观测

门禁必须非零，正常证据仍通过。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

结构化样本阈值与语义检查，避免仅摘要失败掩盖真实检查。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `tools/test_gate.py` 与本章问题具有直接关系。

7. 限制与替代解释

阈值是本地顺序读取教学预算，不能跨硬件作为严格 SLO。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

临时副本与受控坏 PDF 由 TemporaryDirectory 自动清理，原始源和成品不被修改。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E23 远程 CI 失败与恢复

状态：**NOT_RUN**。来源：`.github/workflows/engineering.yml`。证据：`evidence/ci/results.json`。

1. 工程问题与假设

远程门禁能否阻断缺失证据，正常提交能否通过？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

工作流依赖图、真实 runner 退出码与制品。

3. 基线与实验设计

workflow_dispatch 注入缺失 manifest，再按正常输入运行，保留运行 ID。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
gh workflow run engineering.yml --ref codex/enterprise-masterclass-20261111
```

5. 实测观测

以实际 GitHub Actions 结果决定状态，文件存在不算通过。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

工作流依赖图、真实 runner 退出码与制品。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `.github/workflows/engineering.yml` 与本章问题具有直接关系。

7. 限制与替代解释

GitHub 环境与本机不是同一硬件；云部署不在此实验内。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E24 备份恢复机制

状态：**VERIFIED**。来源：`experiments/backup.py`。证据：`evidence/backup/results.json`。

1. 工程问题与假设

pg_dump 自定义格式能否恢复表与中文记录？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

逻辑备份保留模式和数据；恢复目标为独立数据库。

3. 基线与实验设计

备份三行合成日志，pg_restore 到新库并比较精确行内容。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python experiments/backup.py
```

5. 实测观测

三行与原表一致，保存备份字节数与 SHA-256，两库随后清理。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

逻辑备份保留模式和数据；恢复目标为独立数据库。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `experiments/backup.py` 与本章问题具有直接关系。

7. 限制与替代解释

不是完整应用 DR、异机备份或 PITR；没有生产 RTO/RPO。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

finally 只删除此次 source/restore 随机库，二进制备份不持久保留。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E25 移动端工作台

状态：**VERIFIED**。来源：`frontend/tests/workflow.spec.ts`。证据：`evidence/browser/results.json`。

1. 工程问题与假设

390px 视口是否可操作且无横向溢出？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

响应式布局、标签化表单和替代关联路径。

3. 基线与实验设计

390×844 登录、开始调查、查证据和查看图谱，检查 scrollWidth。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
cd frontend
node node_modules/@playwright/test/cli.js test
```

5. 实测观测

流程通过，文档宽度不超过视口，保存真实截图。 当前浏览器结果：4 通过、0 失败。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

响应式布局、标签化表单和替代关联路径。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `frontend/tests/workflow.spec.ts` 与本章问题具有直接关系。

7. 限制与替代解释

没有覆盖所有移动设备或 WCAG 全面审计。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

集成测试仅删除自动创建的 echocase_test_ 数据库；单元测试无外部状态。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

E26 出版校验器反例

状态：**VERIFIED**。来源：`manual/tools/pdfqa.py`。证据：`evidence/qa/pdf-selftest.json`。

1. 工程问题与假设

空白、越界和不匹配目录是否会被拒绝？ 假设：已声明的保护在合法路径允许操作，在负向或竞争条件下给出明确拒绝，不留下半提交状态。

2. 底层机制

PDF 几何、书签映射与两遍稳定性检查。

3. 基线与实验设计

生成受控坏 PDF，分别调用实际 layout 和 TOC 检查命令。 先固定输入、身份、数据与执行条件，再引入一个明确变化；不同时改变多个因素后只归因于其中一个。

4. 执行与复现

从仓库根目录运行。先按 README 配置真实 PostgreSQL 与所需环境变量，使用文档锁定依赖。浏览器实验需要开发 API 与 Vite 已运行；容器实验自行构建。

```
python manual/tools/test_pdfqa.py
```

5. 实测观测

坏 PDF 必须非零；最终成品再跑同一检查。 结论以证据文件中的断言、样本和进程退出码为准；若复现结果不同，保留新结果并将状态降为 PARTIAL。

6. 解释与架构含义

PDF 几何、书签映射与两遍稳定性检查。 因此验收必须检查这一边界上的可观察结果，而不能以组件已安装或代码已生成代替。对应实现 `manual/tools/pdfqa.py` 与本章问题具有直接关系。

7. 限制与替代解释

几何检查不理解图意，仍需人工检查每一页。 单次通过不证明没有其他缺陷；故障注入只覆盖其明确注入的位置与条件。

8. 清理与复核

临时副本与受控坏 PDF 由 TemporaryDirectory 自动清理，原始源和成品不被修改。 对照正常输入再运行一次相关门禁，确认清理没有破坏环境。完整测试与来源应保留在仓库，不将原始日志塞进教材正文。

附录 B 参考来源与工程词汇

一手技术来源

以下来源用于核对框架机制，项目实验则是这些机制在当前配置下的执行证据。在线文档会更新，复现时应同时参考锁文件与本地代码。

主题	官方或上游来源	阅读目的
Clean Architecture	Jason Taylor 源码	DI、请求行为、Identity、事件时机
Cookie API 行为	ASP.NET Core 10 API 认证	401/403 与重定向的边界
CSRF	ASP.NET 防伪文档	Cookie 请求的独立保护
乐观并发	EF Core 并发	原值条件、影响行数与异常
值生成	EF Core 值生成	手动 Guid 与生成约定
PostgreSQL 主键	Npgsql 值生成	Provider 的默认行为
事务隔离	PostgreSQL 17 文档	语句快照、事务快照与序列化失败
乐观界面	TanStack Query 乐观更新	取消、快照、回滚与重新读取
Mutation	TanStack Mutation	写入生命周期与重试
图谱性能	React Flow 性能	节点规模增长时的测量与优化
组件来源	shadcn/ui	可拥有的组件与 MIT 许可证
容器部署设计	Azure 自定义容器	端口、镜像与平台设置；当前 NOT_RUN
云身份设计	App Service 数据库身份	运行身份与数据库权限；当前 NOT_RUN

本项目没有复制参考 DevOps 教材的正文、测量、截图或平台拓扑。采用的是证据分离、门禁自测、两遍目录与逐页审阅的方法。上游审计提交与出处见仓库 THIRD-PARTY-NOTICES.md 和 docs/PLAN.md。

关键术语

术语	在 EchoCase 中的精确定义
Aggregate Root	调查的一致性入口，控制发现、关联、笔记与结论
Invariant	每次合法业务动作完成后必须仍成立的约束
CQRS	命令与查询职责分离；本项目共用单库
Optimistic Concurrency	用原版本作为数据库更新条件，拒绝过期意图
Idempotency Key	标识一次可重试的用户意图，作用域为账户
Receipt	与业务同事务保存的原操作响应快照
MVCC Snapshot	数据库语句或事务所观察的一致可见版本
Write Skew	各自更新不同行仍破坏跨行约束的并发异常
Server State	由服务端权威保存、客户端缓存的调查状态
Dirty Draft	用户已编辑但尚未成功提交的本地内容
Readiness	当前依赖条件下是否可处理需要数据库的工作
Reproducibility	按公开条件执行并获得可比较结果，不承诺字节相同
Evidence Integrity	文件、结构化结果与有限测量来源的一致性检查

复现结果不一致时

先记录版本、命令、退出码与原始输出，再确认输入数据、环境变量、数据库迁移、Cookie 身份与端口。性能差异先检查负载和暖机条件，业务差异先检查预期版本与案件规则。不要为了得到书中数字而改写原始样本，也不要把不同环境的一次失败直接归类为代码问题或环境问题。

实验的价值在于解释变化。若复现揭示新的失败，应新增证据、降低相关状态、修正实现或文字，再重复相同检查。教材与应用共同演化，才能保持技术解释和运行事实一致。